Generative Adversarial Networks for Financial Distributions*

March 19, 2026

Abstract

Stochastic differential equations underpin many models in quantitative finance, yet their numerical simulation is often incompatible with automatic adjoint differentiation (AAD). Exact and high-fidelity schemes frequently rely on rejection or branching mechanisms that break differentiability, while even smooth discretizations may yield unstable or biased gradients. We propose an alternative based on conditional generative adversarial networks (GANs) that learn parametric marginal distributions directly. Implemented as $\mathcal{C}^\infty$ neural networks, the resulting generators are inherently differentiable, provide fixed-cost inference independent of maturity or discretization, and are fully compatible with AAD. Conditional GANs accurately reproduce key distributional features, including mean, variance, skewness, and tail behavior. The distributional distances such as Wasserstein-1 and Kolmogorov-Smirnov remain uniformly low across heterogeneous parameter regimes. Importantly, worst-case errors are controlled: even extreme parameter configurations are fitted reliably, highlighting the robustness induced by the training domain design. We demonstrate the approach on subordinated Lévy models, the Heston model, and the rough Bergomi model. The smooth generators naturally enable Sobolev training of neural pricers using joint price-gradient information.

Keywords: generative adversarial networks, automatic adjoint differentiation, Heston model, subordinated Lévy models, rough Bergomi model.

Mathematics Subject Classification: 62M45, 65C30, 91G20, 91G60, 65C05.
JEL Classification: C15, C63.

1 Introduction

A wide spectrum of foundational models in quantitative finance is built upon stochastic differential equations. The earliest instances include the Bachelier (1900) model and the Black and Scholes (1973) framework, which already illustrate how stochastic dynamics can encode market uncertainty in a mathematically tractable way. Over time, richer structures were introduced to capture empirical features such as volatility clustering, jumps, and long memory. Stochastic volatility models such as Heston (1993), together with subordinated models in which the physical time is replaced by an independent random clock, most notably the Variance Gamma model (Madan and Seneta 1990; Madan, Carr, et al. 1998) and the Normal Inverse Gaussian model (Barndorff-Nielsen 1997; Eberlein and Keller 1995), and more recently rough volatility formulations such as the rough Bergomi model (Bayer, Friz, et al. 2016) or rough Heston model (El Euch and Rosenbaum 2018; El Euch and Rosenbaum 2019), all provide elegant parametrizations and interpretable structural quantities including long run variance, activity rates, and the Hurst exponent.

On the other hand, the ability to compute derivatives has always played a central role in quantitative finance. Sensitivities are required for hedging, for risk management (Huge and Savine 2020), and for calibration procedures that rely on gradient based optimisation (Czarnecki

*Python/Pytorch implementation are available on https://mlp.lpma.math.upmc.fr/GAN_SDE.zip.

et al. 2017). Over the past decades, the emergence of automatic adjoint differentiation (AAD) has profoundly transformed these tasks (Rumelhart et al. 1986; Griewank and Walther 2008; Baydin et al. 2018). In its reverse mode incarnation, AAD delivers gradients at a computational cost essentially independent of the number of parameters, which makes it particularly attractive for high dimensional models. Its successful application however relies on a computational graph that is both static and differentiable, so that every operation in the simulation pipeline can be propagated backward in a stable manner. From this perspective, it would be highly desirable to reconcile the numerical methods used to simulate financial models with the smoothness and structural requirements of AAD.

Unfortunately, many exact or high fidelity simulation techniques that generate samples from the laws of stochastic models are pathologically incompatible with this framework. Rejection methods, as illustrated for instance by the algorithm of Marsaglia and Tsang (2000), introduce abrupt accept or reject decisions that create non smooth discontinuities in the computational graph. Branching loops of the type appearing in the work of Michael et al. (1976) similarly rely on conditional path dependent logic that alters the sequence of operations in an unpredictable manner. Both mechanisms violate the requirement of a static and differentiable graph, and therefore obstruct the use of reverse mode automatic adjoint differentiation in a robust and reliable way.

Even when numerical schemes could remain fully compatible with AAD when involving no explicit sources of non-smoothness and producing a stable computational graph, substantial difficulties still persist. Classical discretizations produce very long chains of operations that are susceptible to vanishing, exploding, and shattering gradients (Bengio et al. 1994; Glorot and Bengio 2010; Balduzzi et al. 2017; Ioffe and Szegedy 2015; He et al. 2016). More fundamentally, the convergence of a numerical scheme toward the true solution of a stochastic differential equation does not necessarily imply the convergence of its associated parameter sensitivities (Hager 2000; Hückelheim et al. 2024). Recent analyses of stochastic adjoint methods by X. Li et al. (2020) indicate that gradient convergence requires uniform stability properties of the discrete flow that are not guaranteed by strong convergence results. Thus, numerical simulation techniques that are otherwise standard in quantitative finance may fail to provide reliable derivatives, which complicates the integration of AAD into model generation pipelines.

Neural networks, by contrast, are inherently compatible with AAD thanks to their smooth computational structure, as long as we use a smooth activation function. Moreover, a wide spectrum of approaches has emerged in recent years, from variational autoencoders (Kingma and Welling 2013) to normalizing flows (Kobyzev et al. 2020) and diffusion models (Ho et al. 2020), with surveys providing comparative assessments across families (Bond-Taylor et al. 2021). In this landscape, we focus on generative adversarial networks (GANs) (Goodfellow et al. 2014) and their conditional variants (Mirza and Osindero 2014). Their particular appeal lies in the combination of high fidelity to target distributions with extremely fast inference, a balance less naturally attained by other paradigms (Bond-Taylor et al. 2021).

In finance, GANs have already been used successfully for time series generation, for instance in the Quant GANs framework (Wiese et al. 2020) or in conditional settings leveraging Neural SDEs with signature-based distances (Lozano et al. 2023). Beyond purely data-driven sequence modeling, recent work has extended adversarial training to the simulation of stochastic differential equations, either by designing supervised GANs that approximate strong solutions to classical diffusions (Rhijn et al. 2023), or by interpreting Neural SDEs directly as continuous-time GANs (Kidger et al. 2021).

In this work, however, we deliberately remain in a weaker sense: our goal is to train conditional GANs whose generator takes as input the parameters of a financial model, maturity included, and outputs directly the marginal distribution. Once trained, such a generator, implemented here as a $\mathcal{C}^\infty$ neural network, offers three decisive advantages. First, its smooth structure gives direct access to parameter sensitivities through AAD, readily supported in libraries such

as PyTorch. Second, inference cost becomes entirely fixed: it depends only on the dimension of the input and the architecture of the network, and is independent of maturity or discretization choices. Third, neural networks align perfectly with GPU architectures, allowing forward sampling to be massively parallelized with near-optimal efficiency (Shi et al. 2016; Y. E. Wang et al. 2019; Castelló et al. 2022). The conceptual shift is therefore clear: the smooth generator itself becomes a new, differentiable model that captures the full distributional behavior of the training dynamics and is ready for direct use.

A natural application of these generators is the *Sobolev training* (ST) (Czarnecki et al. 2017). Rather than fitting a pricer solely on option prices, the network is trained to interpolate both prices and their sensitivities with respect to the model parameters. This distinction is crucial: a classical L^2 training on prices alone generally fails to deliver accurate sensitivities, even if the fit on prices appears satisfactory (Czarnecki et al. 2017). Yet sensitivities are every bit as important as prices in finance (Huge and Savine 2020). With classical simulators, incorporating sensitivities into training would be perilous or even infeasible as discussed below. In contrast, the GAN generator naturally provides parameter–payoff–gradient triplets via AAD, which makes ST fully applicable. This yields differentiable pricers trained on data generated by the GAN generators themselves, and directly aligned with the requirements of quantitative finance.

In this work, we develop and evaluate such generators across three representative families: subordinated models with the Normal Inverse Gaussian (NIG) Barndorff-Nielsen (1997) and VG models, the Heston model, and the rough Bergomi model. These models are chosen for their contrasting distributional shapes, volatility structures, and memory effects. Our purpose is to demonstrate that conditional GANs can successfully learn financial parametric distributions, provided that parameter domains are carefully designed to balance financial realism with numerical stability. Despite the well-known challenges of adversarial training (Goodfellow et al. 2014; Arjovsky et al. 2017; Z. Li et al. 2022) and the spectral bias of neural networks (Rahaman et al. 2019), performance remains consistent across all four studied models. Conditional GANs reproduce mean, variance, skewness, and tail behavior with high accuracy. The *mean preserving property* is satisfied within 10^{-3}, and distributional distances (Wasserstein-1 and Kolmogorov-Smirnov) are uniformly low even across heterogeneous regimes. Importantly, worst-case errors remain controlled: even extreme parameter configurations are fitted reliably, underlining the robustness of the training domain design.

The key perspective of this contribution, however, is to view the generator not simply as a surrogate for simulation, but as a new model in its own right. It encapsulates the full distributional richness of the training dynamics within a $\mathcal{C}^\infty$ parametric mapping, inherently differentiable and immediately exploitable through AAD. In this guise, it adapts particularly well to ST: by design, the generator provides seamless access to parameter-price-gradient triplets, making it natural to train highly accurate neural pricers on both prices and sensitivities.

Section 2 introduces automatic adjoint differentiation (AAD) and highlights its incompatibilities with classical simulation methods, particularly those relying on accept–reject mechanisms or branching logic. Section 3 then presents generative adversarial networks (GANs) (Goodfellow et al. 2014) and their conditional variants (Mirza and Osindero 2014). In Section 3, we establish a testing protocol to benchmark distributional fidelity using moments and metrics such as the Wasserstein-1 distance and the Kolmogorov–Smirnov statistic (Ramdas et al. 2017). Building on this foundation, Section 4 reports training results for models driven by subordinators, VG and NIG. It is complemented by Section 4.3, which focuses on reconstructing discrete-time trajectories from these marginal generators. Section 5 then turns to the Heston model, with a dedicated subsection 5.3 addressing the same trajectory reconstruction problem, and Section 6 presents our results on the rough Bergomi model. In the latter case, since a marginal generator cannot be exploited to reconstruct full trajectories in the rough setting, it can only be trained to learn the law between the initial time t_0 and maturity T, thus providing a tractable albeit partial surrogate. Finally, Section 7 introduces the ST methodology and illustrates its applica-

tion to the generator trained on the Heston model, exploiting the $\mathcal{C}^\infty$ nature of the generator to construct joint price–gradient datasets.

2 Automatic adjoint differentiation and simulation

2.1 Principles and motivation

Automatic differentiation (AD) (Rumelhart et al. 1986; Griewank and Walther 2008) is a computational technique that provides exact (machine-precision) gradients by recording a function evaluation as a computational graph and systematically applying the chain rule. Two main modes exist: forward-mode AD and backward-mode AD. The latter is particularly efficient when functions have few outputs (typically a scalar) but many inputs, as it computes all partial derivatives simultaneously with a single backward sweep. This makes it ideally suited for high-dimensional parameter spaces, such as neural network training via backpropagation or the evaluation of pricing and risk functionals in quantitative finance (Homescu 2011)

Remark 2.1. The terminology varies across fields. In the algorithmic differentiation literature, the umbrella term is *algorithmic (or automatic) differentiation* (AD), with two main modes: forward and backward. In quantitative finance, however, the backward mode is almost universally referred to as *adjoint algorithmic (or adjoint automatic) differentiation* (AAD). Forward-mode AD remains better suited to problems with few inputs and many outputs.

From graphs to gradients As detailed by Gremse et al. (2016), AAD represents a computation as a *directed acyclic graph* (DAG) of elementary functions and can be viewed as the interaction of two complementary pipelines. The first pipeline constructs a *computation graph* of elementary functions f_i, where each node records the local derivative rules associated with its operation. The second pipeline performs the forward evaluation, storing only the intermediate values produced by each f_i.

Once both are available, the backward pass consists in applying the stored values to the derivative rules kept in the graph, propagating adjoints from the output back to the input to obtain exact total derivatives. Because the computation graph itself can be preserved, subsequent calls to the same function f need only record intermediate values; the pre-built graph then drives the backward propagation.

2.2 Limitations in common simulation methods

AAD assumes that the computation graph is (i) *composed of differentiable operations* and (ii) *structurally static*, i.e. the same fixed sequence of operations applies to all samples. The first requirement is mathematical: each operation must admit well-defined derivatives. The second limitation is practical: in autodiff frameworks, one ideally works with a fixed gradient—like a true derivative that can be evaluated directly as a function of the inputs. In principle, one could construct dynamic graphs that adapt to random draws or parameter regimes, but this is computationally expensive and memory-intensive. Moreover, dynamic graphs often embed globally non-differentiable steps, such as accept–reject sampling.

Several standard simulation methods violate these assumptions. In what follows, we summarise three of them:

- **Rejection sampling and random-length loops.** Accept–reject algorithms introduce loops where the number of iterations depends on random draws. This makes the execution path random, which autodiff engines cannot follow reliably. For instance, in a typical accept–reject scheme one draws $U \sim \mathcal{U}(0,1)$ until acceptance:

$$\texttt{while } U > p(x): \quad U \sim \mathcal{U}(0,1),$$

where x is a fixed input of the function. Even when called twice on the same input x, the function may trace two entirely different computational graphs depending on how many rejections occur. This variability is undesirable, because the computation graph should remain stable for a fixed input. Classical algorithms such as Marsaglia and Tsang (2000) for the Gamma distribution rely on rejection steps, and thus produce computational graphs that differ across realizations.

- **Conditional branches and non-smooth operators.** As rejection sampling, where the number of repetitions is random, conditional branching makes the computation path itself random. Explicit `if/else` rules select different formulas depending on draws or parameter values, creating non-differentiable boundaries. *Example:*

$$U \sim \mathcal{U}(0,1), \quad f(x) = \begin{cases} \cos(x), & U < 0.5, \\ \sin(x), & U \geq 0.5, \end{cases}$$

  so the function applied to x depends on a random draw U, making the gradient path inconsistent across simulations. Even continuous but non-smooth operators, such as $\max(0, x)$, may produce undefined or highly unstable gradients. A typical example of conditional branching is provided by the algorithm of Michael et al. (1976) for the Inverse Gaussian distribution. Like random-length loops, the underlying computational graph changes from one call to the next, making AAD unreliable. In practice, one may still obtain gradients because many libraries assign an artificial derivative to non-smooth points (e.g. $\max(0, x)$ at $x = 0$), but these values are fragile numerical conventions rather than theoretically valid derivatives.

- **Singularities and floating-point stability.** Some operations yield derivatives that either blow up or vanish near critical points. For instance,

$$\frac{d}{dx}\sqrt{x} = \frac{1}{2\sqrt{x}} \;\to\; \infty \quad \text{as } x \to 0,$$

  leads to gradient explosion. Conversely, other transformations drive derivatives arbitrarily close to zero, causing vanishing gradients. In principle, such behaviours pose no mathematical difficulty: the chain rule remains valid. In floating-point arithmetic, however, repeated multiplications of unstable derivatives accumulate round-off errors, producing overflow or underflow. This phenomenon is well documented in the numerical analysis literature, from the pioneering work of Wilkinson on rounding errors (Wilkinson 1963), to Goldberg's influential survey on floating-point arithmetic (Goldberg 1991), and Higham's modern treatment of stability and accuracy (Higham 2002).

- **Discretization and numerical schemes.** A first limitation arises from the depth of the computational graph. In numerical schemes based on small time steps, the reverse mode chain rule must propagate through a very long sequence of Jacobian multiplications. Such deep computation chains exhibit the same instabilities as those encountered in deep learning architectures: gradients may vanish, explode, or become effectively random through shattering effects (Bengio et al. 1994; Glorot and Bengio 2010; Balduzzi et al. 2017; Ioffe and Szegedy 2015; He et al. 2016). These phenomena already impair the stability and accuracy of reverse mode differentiation in discretized stochastic dynamics.

  Beyond these stability issues, a more fundamental limitation is that AAD differentiates the implemented program rather than the underlying continuous model. Even with smooth operations and a static computation graph, discretizing a stochastic differential equation alters the flow of sensitivities. The review of automatic differentiation pitfalls by Hückelheim et al. (2024) stresses that discretized models may approximate the state dynamics of a continuous system while producing derivatives that bear little relation to those of the true

flow. This phenomenon is consistent with the classical analysis of Hager (2000), which shows that discretizing a differential system and discretizing its adjoint do not necessarily commute. It is further reinforced in the stochastic setting, where X. Li et al. (2020) demonstrate that convergence of a numerical scheme does not imply convergence of its parameter sensitivities. Consequently, both the instability of deep computational chains and the unreliability of discrete derivatives limit the effectiveness of naive automatic adjoint differentiation for complex numerical schemes.

These generic obstacles manifest differently across classical models:

Heston. Vostrikova (2009) and Abbas-Turki and Lamberton (2014) show that the stochastic flow of the CIR process (Cox et al. 1985),

$$dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dZ_t,$$

remain well-behaved only under the Feller condition $\delta = 2\kappa\bar{v}/\gamma^2 > 1$. In addition, the square-root term itself yields exploding gradients near zero, notably when the Feller condition is not satisfied. These structural instabilities comes on top of the documented dangers of applying AAD to numerical schemes (Hager 2000; X. Li et al. 2020; Hückelheim et al. 2024), making AAD on discretized CIR dynamics essentially impractical.

Beyond this analytical issue, exact simulation schemes for the CIR law—whether based on noncentral-χ^2 sampling, Poisson–Gaussian mixtures, Laplace inversion, or tabulation—introduce conditional branches and random-length loops. These algorithmic constructs disrupt the static computation graph assumed by AAD frameworks, rendering backward-mode sensitivities unreliable even in regimes where the variance remains strictly positive.

Variance Gamma and Normal Inverse Gaussian models. Both models subordinate Brownian motion to an independent clock G_t. Exact simulation of G_t relies on accept–reject procedures for the Gamma subordinator (Marsaglia and Tsang 2000) or on conditional branching for the Inverse Gaussian (Michael et al. 1976), both of which are incompatible with AAD. Even if differentiable generation schemes were available for the subordinators, the composition with Brownian motion would remain problematic, since sample paths of Brownian motion are almost surely nowhere differentiable in time.

Rough Bergomi: The rough Bergomi model defines forward variance through a fractional Volterra field,

$$\widetilde{W}_t = \int_0^t (t-s)^{H-\frac{1}{2}}\,dW_s, \qquad H \in (0, \tfrac{1}{2}),$$

which is non-Markovian and, for $H < \frac{1}{2}$, not a semimartingale (Bayer, Friz, et al. 2016). Numerical simulation is notoriously demanding: the singular kernel forces very fine time discretizations to achieve convergence (Friz et al. 2025), while the generation of fractional Brownian trajectories is itself computationally costly (Bennedsen et al. 2017; Abi Jaber and El Euch 2019). As a result, standard schemes generate extremely deep computation graphs. In such settings, one risks encountering the same pathologies as in deep learning, where gradients in very deep graphs tend to vanish or explode (Bengio et al. 1994; Glorot and Bengio 2010). And even if such instabilities could be alleviated, there is still no guarantee that the gradients of discretized schemes faithfully represent those of the underlying continuous model, as discussed in Hager (2000), X. Li et al. (2020), and Hückelheim et al. (2024).

The guiding idea of this chapter is to train $\mathcal{C}^\infty$ generative models that reproduce the marginal laws of classical processes. Such a generator can then be interpreted in two complementary ways.

First, as a surrogate simulator: it approximates the reference process less accurately than very fine-grained numerical schemes, but has the decisive advantage of being fully differentiable. Second, as a model in its own right: once trained, it defines a new stochastic specification whose gradients with respect to parameters are immediately accessible through AAD. Neural networks naturally avoid all the pitfalls that hinder the application of AAD to classical simulation methods (see Table 1), with the exception of vanishing and exploding gradients. This latter issue is more nuanced, but in practice the risk remains low when training shallow architectures, typically with less than four hidden layers.

Table 1: Obstacles for AAD and how generators compare.

Obstacle	**In numerical simulators**	**In generators ($\mathcal{C}^\infty$)**
singularities	Exploding / vanishing gradients	More stable by Spectral Bias and $\mathcal{C}^\infty$ structure
Random-length loops	Accept–reject steps with stochastic iteration count	Absent: fixed-depth evaluation
Conditional branches	`if/else` logic, non-smooth operators	Absent: smooth functional map
Deep graphs	Many time steps $\Rightarrow$ long gradient chains	Shallow feed-forward network $\Rightarrow$ short gradient chains

3 Generative adversarial networks

3.1 Conditional parametric generation

Generative Adversarial Networks (GANs) (Goodfellow et al. 2014) set up a minimax game between two players. Let Z be a latent random variable taking values in $\mathbb{R}^d$, and let $G : \mathbb{R}^d \to \mathbb{R}^n$ denote a neural network (the generator) that maps $z \in \mathbb{R}^d$ to a synthetic output in $\mathbb{R}^n$. We denote by

$$X^G := G(Z)$$

the random variable generated by G when fed with noise Z. On the other side, the true distribution is represented by a target random variable X^{true} with values in $\mathbb{R}^n$. A discriminator $D : \mathbb{R}^n \to [0, 1]$ is trained to distinguish between generated samples X^G and real samples X^{true}, while G is trained to fool D. The original objective is

$$\min_G \max_D \ \mathbb{E}\big[\log D(X^{true})\big] + \mathbb{E}\big[\log\big(1 - D(X^G)\big)\big] \,,$$

which minimises the Jensen–Shannon divergence but often suffers from vanishing gradients when the supports of real and generated data do not overlap (Arjovsky et al. 2017). More broadly, GAN training can be unstable, with oscillations or *mode collapse* (Z. Li et al. 2022), especially in high-dimensional and multimodal domains such as images. Our setting is much simpler: we model one-dimensional conditional laws—returns or log-returns—under parametric models (Heston, VG, NIG, rough volatility). These distributions are low-dimensional, which greatly mitigates classical GAN pathologies and enables stable training with modest architectures, here implemented as shallow networks with only four hidden layers.

In its basic form, a GAN can be understood as the generator of a single marginal distribution but the principle can be extended to an entire parametric family $\{X^{true}(\Theta) : \Theta \in \mathcal{D}\}$, where $\mathcal{D} \subset \mathbb{R}^p$ is a compact parameter domain. The generator is then defined as

$$G : \mathbb{R}^p \times \mathbb{R}^d \to \mathbb{R}, \qquad (\Theta, z) \mapsto G(\Theta, z),$$

and we denote

$$X^G(\Theta) := G(\Theta, Z),$$

with latent noise Z. The objective is to ensure that, for all $\Theta \in \mathcal{D}$,

$$X^G(\Theta) \overset{d}{=} X^{true}(\Theta).$$

Formally, let $\Xi \sim \mathcal{U}(\mathcal{D})$ be independent of Z. Training consists in enforcing that $(\Xi, X^G(\Xi))$ and $(\Xi, X^{true}(\Xi))$ have the same joint law, through the adversarial objective

$$\min_G \max_D \mathbb{E}\Big[\log\Big(D\big(X^{true}(\Xi), \Xi\big)\Big)\Big] + \mathbb{E}\Big[\log\Big(1 - D\big(X^G(\Xi), \Xi\big)\Big)\Big].$$

In practice, given a batch of size N, we draw parameters $(\Theta_i)_{i=1}^N \sim \mathcal{U}(\mathcal{D})$, then sample $x_i^{true} \sim X^{true}(\Theta_i)$, $z_i \sim Z$, and set $x_i^G := G(\Theta_i, z_i)$. The empirical loss is

$$\mathcal{L}(G, D) \;=\; \frac{1}{N}\sum_{i=1}^{N}\Big[\log D(x_i^{true}, \Theta_i) + \log\big(1 - D(x_i^G, \Theta_i)\big)\Big].$$

This framework can be interpreted as a particular instance of *conditional* GANs (cGANs) (Mirza and Osindero 2014). In the original formulation, conditioning is typically provided by external labels, such as class annotations in image datasets. Here, the conditioning variable is the parameter vector Θ, which fully determines the target distribution $X^{true}(\Theta)$. Figure 1 illustrates the mechanism of the cGAN.

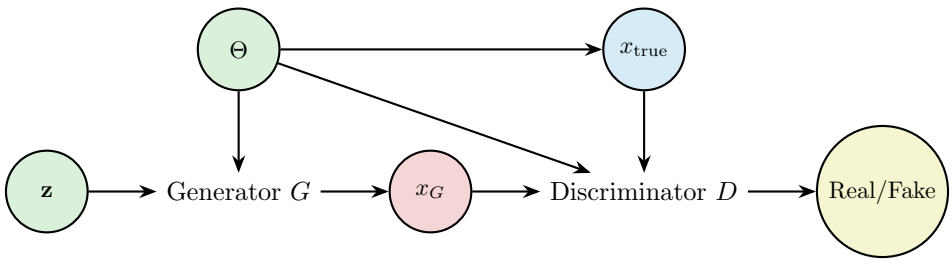


Figure 1: Structure of a conditional GAN with a generator taking noise $\mathbf{z}$ and parameters Θ and produce a synthetic sample x_G and with a discriminator observing (Θ, x) were $x \in \{x_G, x_{\text{true}}\}$ and classifies them as real or fake.

This parametric conditioning turns the trained generator into a differentiable, parameter-aware simulator with three key advantages for finance:

1. **Structural conditioning.** Injecting Θ during training ensures that each parameter vector maps to its correct conditional distribution.

2. **Amortised simulation.** After training, $G(\Theta, z)$ generates i.i.d. samples at fixed cost, regardless of maturities or path-construction details.

3. **Smoothness.** With smooth activations, the learned map $(\Theta, z) \mapsto x$ is $\mathcal{C}^\infty$ w.r.t Θ, so gradients exist and are directly available through standard backpropagation.

We now detail the network architectures used for the adversarial game.
Generator. The generator is a mapping

$$G : \mathbb{R}^p \times \mathbb{R}^d \to \mathbb{R}, \qquad (\Theta, z) \mapsto G(\Theta, z).$$

Linear embeddings. We first embed Θ and z via affine maps

$$h_\Theta = W_\Theta \Theta + b_\Theta, \quad W_\Theta \in \mathbb{R}^{m_\Theta \times p},\ b_\Theta \in \mathbb{R}^{m_\Theta},$$

$$h_z = W_z z + b_z, \quad W_z \in \mathbb{R}^{m_z \times d},\ b_z \in \mathbb{R}^{m_z},$$

so that $h_\Theta \in \mathbb{R}^{m_\Theta}$ and $h_z \in \mathbb{R}^{m_z}$. We concatenate these two embeddings into

$$c = \left[\, h_\Theta \parallel h_z \,\right] \in \mathbb{R}^{m_\Theta + m_z}.$$

A multilayer perceptron (MLP) (Hornik et al. 1989) is defined as a composition of affine maps and nonlinear activations. Formally, for an input $x \in \mathbb{R}^n$,

$$\psi(x) = A_{L_\psi} \circ \sigma \circ A_{L_\psi - 1} \circ \cdots \circ \sigma \circ A_1(x),$$

where each $A_l(x) = W_l x + b_l$ is affine and the activation σ acts elementewise.

The MLP used in our generator architecture, denoted ψ_G, contains at least three affine maps. Its weight matrices have dimensions

$$W_1 \in \mathbb{R}^{m_G \times (m_\Theta + m_z)}, \qquad W_l \in \mathbb{R}^{m_G \times m_G}\ (2 \le l \le L_G - 1), \qquad W_{L_G} \in \mathbb{R}^{1 \times m_G}.$$

so that the generator output is a scalar,

$$G(\Theta, z) = \psi_G(c) \in \mathbb{R}.$$

Here, m_G denotes the *hidden width*.

Discriminator. The discriminator $D : \mathbb{R} \times \mathbb{R}^p \to [0, 1]$ has the same architecture as the generator, except that instead of a noise vector $z \in \mathbb{R}^d$, a scalar $y \in \mathbb{R}$ is embedded into $\mathbb{R}^{m_y}$. Concretely,

$$g_\theta = W'_\Theta\, \Theta + b'_\Theta, \qquad W'_\Theta \in \mathbb{R}^{m_\Theta \times p},\ b'_\Theta \in \mathbb{R}^{m_\Theta},$$

$$g_y = W_y\, y + b_y, \qquad W_y \in \mathbb{R}^{m_y \times 1},\ b_y \in \mathbb{R}^{m_y}.$$

The embeddings are concatenated into

$$s = [\, g_y \parallel g_\Theta \,] \in \mathbb{R}^{m_\Theta + m_y},$$

and passed through an MLP $\psi_D : \mathbb{R}^{m_\Theta + m_y} \to \mathbb{R}$ with *hidden width* m_D and L_D linear layers. A final logistic sigmoid

$$D(y, \Theta) = \sigma(\psi_D(s)), \qquad \sigma(x) = \frac{1}{1 + e^{-x}},$$

ensures that the output lies in $[0, 1]$ and can be interpreted as a probability.

As activation, we use the smooth Softplus

$$\sigma_\beta(x) = \tfrac{1}{\beta} \log\!\big(1 + e^{\beta x}\big),$$

for both generator and discriminator. For numerical stability in PyTorch, when βx is large the exponential form of Softplus is replaced by its linear asymptote. We use $\beta = 10$, large enough to behave as a numerically C^∞ "smoothed ReLU" on the relevant range. This activation is employed consistently in both the generator and the discriminator networks. For clarity, we characterize each network by its **parameters**, which specifies the sizes of input embeddings, hidden layers, depth, and output dimension:

$$\text{Generator } G:\ (p, d, m_\Theta, m_z, m_G, L_G), \qquad \text{Discriminator } D:\ (p, m_\Theta, m_y, m_D, L_D).$$

Note that both networks share the same parameter embedding dimension m_Θ, ensuring a consistent representation of model parameters across generator and discriminator.

Training setup. All models are trained with the standard GAN loss using the Adam optimizer (Kingma and Ba 2015), with learning rate 10^{-4} and exponential decay 0.99. For each generator update, the discriminator is updated five times. Training runs for 1000 epochs with batch size 2000 in `float32` on GPU. The validation phase is performed by averaging Wasserstein distances between the generated distribution and the true law, evaluated on 20 000 independent parameter configurations. Although 20 000 may seem small compared to the 5 million training configurations, each Wasserstein distance is itself approximated using 10 000 real and generated samples, which already accounts for the majority of the total training time. A balance had therefore to be struck between statistical robustness and computational feasibility.

On the importance of parameter ranges. A central factor is the choice of the training parameter domain Θ. While the theoretical objective is to learn a generator $G(\Theta, z)$ that generalises across the full space, it is rarely advisable to expose the network to excessively heterogeneous regimes. In models such as VG, NIG, or Heston, parameter sets may correspond to conditional laws $p(x \mid \Theta)$ with very different behaviours, for instance from light- to heavy-tailed, or from near-symmetric to strongly skewed. Asking a single feedforward network to cover such diverse dynamics risks diluting its representational capacity (Rahaman et al. 2019). For this reason, we favour restricting Θ to regime-relevant subspaces, which balances realism with numerical stability. When broader coverage is needed, partitioning Θ into subdomains with specialised generators, a mixture-of-experts approach (Jordan and Jacobs 1994; Chen et al. 2022), offers robustness, at the cost of some stitching overhead.

Remark 3.1. In all models considered here (VG, NIG, Heston, rough Bergomi), the initial spot price S_0 does not appear explicitly as a conditioning parameter. Indeed, it suffices to simulate under the normalised convention $\tilde{S}_0 = 1$, since trajectories or marginal laws for any S_0 can then be obtained by a simple rescaling $S_t = S_0\,\tilde{S}_t$. Excluding S_0 from the parameter vector thus lightens the conditioning input without any loss of generality.

3.2 Evaluation protocol after training

The purpose of evaluation is to check how close the generative law is to the target distribution. To do so, we run several complementary tests (*mean preserving property*, moment matching, and distributional distances) by comparing generated samples with reference simulations on a large grid of parameters. We draw $N = 100{,}000$ parameter vectors $\{\Theta_i\}_{i=1}^N$ uniformly from the admissible domain. For each Θ_i, we generate 100,000 samples

$$X_i^G = G(\Theta_i, z_i).$$

Since the raw generator outputs are not directly expressed as tradable prices, we post-process them into S^G. For *log-models* (Heston, rough Bergomi), X^G is interpreted as log-prices and we set

$$S_i^G = \exp\big(X_i^G\big).$$

For *raw-models* (VG, NIG), X^G is interpreted as the Lévy process itself, and we obtain prices as

$$S_i^G = \exp\big(X_i^G - w(\Theta_i)\big),$$

where $w(\Theta)$ is the martingale adjustment of the true model (see Section 4).

In parallel, we simulate benchmark prices S_i^{true} under the exact model. All evaluations are therefore carried out on the two price distributions S^G and S^{true}, which ideally, should satisfy the *mean-preserving property*: starting from $S_0 = 1$, their expectation remains equal to 1 with negligible variance across Θ_i.

Remark 3.2. In practice, this need not hold exactly: for instance, the Heston diffusion may fail to be a true martingale and only remain a positive local martingale (Keller-Ressel 2011), while subordinated models such as VG or NIG can approach pathological regimes, e.g. large values of κ and θ, see Figure 3). Nevertheless, the empirical results reported in Subsections 4.2, 5.2, and 6.2 show average values extremely close to 1, so that the term *mean-preserving property* can safely be assumed in what follows.

To quantify distributional fidelity, we rely on two classical distances. Let F_i^G and F_i^{true} denote the cumulative distribution functions (CDFs) of the post-processed prices S_i^G and S_i^{true}, respectively. for The Kolmogorov–Smirnov statistic is defined by

$$\text{KS}(F_i^G, F_i^{\text{true}}) = \sup_{x\in\mathbb{R}} \left|F_i^G(x) - F_i^{\text{true}}(x)\right|,$$

and captures the largest local discrepancy between the two CDFs. The 1-Wasserstein distance is given by

$$W_1(F_i^G, F_i^{\text{true}}) = \int_0^1 \left|(F_i^G)^{-1}(u) - (F_i^{\text{true}})^{-1}(u)\right| du,$$

and measures the average "mass transport" needed to match one distribution with the other. Together, these two metrics provide complementary perspectives: KS highlights the worst-case local deviation, while Wasserstein captures global distributional fidelity (Ramdas et al. 2017). We note

$$\text{KS}_i = KS(F_i^G, F_i^{\text{true}}), \qquad W_{1,i} = W_1(F_i^G, F_i^{\text{true}}).$$

Quantities of interest. For each parameter set Θ_i, we compute conditional means and standard deviations,

$$\mu_i^{\text{true}} = \mathbb{E}[S_i^{\text{true}}], \quad \mu_i^G = \mathbb{E}[S_i^G], \qquad \sigma_i^{\text{true}} = \text{Std}[S_i^{\text{true}}], \quad \sigma_i^G = \text{Std}[S_i^G].$$

The notation $\mathbb{E}[\cdot]$ is used in a context-dependent manner. When referring to theoretical quantities, $\mathbb{E}$ denotes the expectation with respect to the underlying probability law. In contrast, for empirical metrics used here $\mathbb{E}$ stands for the empirical mean computed over a finite sample of realizations. This unified notation is adopted for conciseness, with the precise interpretation being clear from the context.

Since $\mathbb{E}[S_i^{\text{true}}]$ is expected to remain very close to 1, see Remarks 3.1 and 3.2, we also define

$$\bar{\mu}_i^{\text{true}} = \mu_i^{\text{true}} - 1, \qquad \bar{\mu}_i^G = \mu_i^G - 1,$$

which measure departures from the theoretical mean.

Absolute and relative errors are then given by

$$e_i^\mu = |\mu_i^G - \mu_i^{\text{true}}|, \quad \tilde{e}_i^\mu = \frac{|\mu_i^G - \mu_i^{\text{true}}|}{|\mu_i^{\text{true}}|}, \qquad e_i^\sigma = |\sigma_i^G - \sigma_i^{\text{true}}|, \quad \tilde{e}_i^\sigma = \frac{|\sigma_i^G - \sigma_i^{\text{true}}|}{\sigma_i^{\text{true}}}.$$

Aggregation. For each quantity q_i, the result tables report the triplet

$$\left(\mathbb{E}[q_i],\ \text{std}(q_i),\ \sup_i |q_i|\right),$$

i.e. its mean, standard deviation, and worst-case deviation across all N parameter sets. The precise correspondence is summarised below:

	$\bar{\mu}_i^{\text{true}}$	$\bar{\mu}_i^G$	e_i^μ	$\tilde{e}_i^\mu$	e_i^σ	$\tilde{e}_i^\sigma$	KS_i	$W_{1,i}$
$\mathbb{E}[\cdot]$	$\mathbb{E}[\bar{\mu}_i^{\text{true}}]$	$\mathbb{E}[\bar{\mu}_i^G]$	$\mathbb{E}[e_i^\mu]$	$\mathbb{E}[\tilde{e}_i^\mu]$	$\mathbb{E}[e_i^\sigma]$	$\mathbb{E}[\tilde{e}_i^\sigma]$	$\mathbb{E}[\text{KS}_i]$	$\mathbb{E}[W_{1,i}]$
$\text{std}(\cdot)$	$\text{std}(\bar{\mu}_i^{\text{true}})$	$\text{std}(\bar{\mu}_i^G)$	$\text{std}(e_i^\mu)$	$\text{std}(\tilde{e}_i^\mu)$	$\text{std}(e_i^\sigma)$	$\text{std}(\tilde{e}_i^\sigma)$	$\text{std}(\text{KS}_i)$	$\text{std}(W_{1,i})$
$\sup_i \lvert\cdot\rvert$	$\sup_i \lvert\bar{\mu}_i^{\text{true}}\rvert$	$\sup_i \lvert\bar{\mu}_i^G\rvert$	$\sup_i e_i^\mu$	$\sup_i \tilde{e}_i^\mu$	$\sup_i e_i^\sigma$	$\sup_i \tilde{e}_i^\sigma$	$\sup_i \text{KS}_i$	$\sup_i W_{1,i}$

The first column, $\bar{\mu}_i^{\text{true}}$, serves as a sanity check: it confirms that the benchmark simulator preserves the mean $\mathbb{E}[S_i^{\text{true}}] = 1$, see Remarks 3.1 and 3.2. The second column, $\bar{\mu}_i^G$, is more revealing: the generator must reproduce this property with the same level of accuracy. The other metrics speak to a different level of fidelity. They are not about anchoring the mean, but about proving that the generator has captured the full shape of the law: its dispersion, its skew, its heavy tails. In this sense, the GAN emerges not merely as a surrogate, but as a probabilistic model in its own right, as expressive as the training law and, crucially, endowed with the indispensable *mean preserving property*.

4 Generative networks for subordinated Lévy models

4.1 Data Generation

The first models on which we train our generative networks are subordinated Lévy processes (Sato 1999), which provide a unified and tractable framework for modelling asset returns with skewness and excess kurtosis. They are constructed by randomising the physical time t of a Brownian motion with drift through an independent, non-decreasing process G_t (a *subordinator*), yielding the time-changed process

$$X_t \;=\; \theta\, G_t + \sigma\, W_{G_t}, \qquad G_t \geq 0,\; W \text{ independent of } G$$

where $\theta \in \mathbb{R}$ and $\sigma > 0$ control drift and diffusion, while the law of G_t governs jump activity, temporal clustering, and tail behaviour of X_t. In this way, one retains analytical tractability (via characteristic functions) while accommodating rich non-Gaussian features.

Two subordinators have proved especially influential in finance. The first is the Gamma process, introduced by Madan and Seneta (1990) in the VG model, defined by

$$G_t \sim \text{Gamma}(t/\kappa,\, 1/\kappa)\,, \qquad \mathbb{E}[G_t] = t, \quad \text{Var}(G_t) = \kappa t,$$

so that small κ yields paths close to Brownian motion, while larger values induce greater effective jump activity, skewness, and tail thickness (Figure 2a). A closely related process is the Inverse Gaussian (IG) process, which underlies the NIG model (Barndorff-Nielsen 1997; Eberlein and Keller 1995) with

$$G_t \sim \text{IG}\big(\mu = t,\, \lambda = t^2/\kappa\big)\,, \qquad \mathbb{E}[G_t] = t, \quad \text{Var}(G_t) = \kappa t,$$

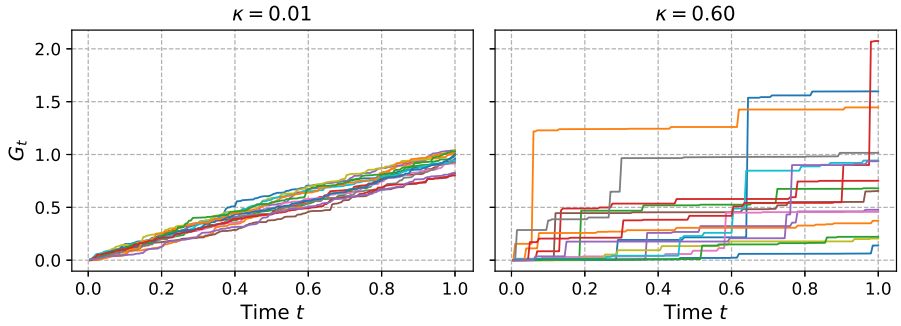
the resulting time change accentuates temporal clustering and produces heavier tails than in the Gamma case (Eberlein and Prause 2002) (Figure 2b).

Remark 4.1. Subordinators are parametrised so that $\mathbb{E}[G_T] = T$, ensuring temporal comparability across models.

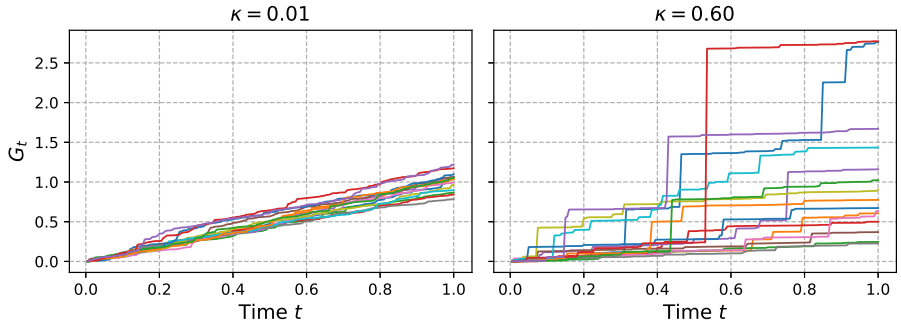
Training the generative model requires large-scale samples of these subordinated processes, obtained via exact simulation of

$$X_T \;=\; \theta\, G_T \;+\; \sigma\sqrt{G_T}\,\xi, \qquad \xi \sim \mathcal{N}(0,1), \qquad G_T \sim \text{Subordinator}, \quad \xi \text{ independent of } G_T.$$

In the VG specification, G_T follows a Gamma distribution which can be sampled using classical hybrid schemes: Marsaglia and Tsang (2000) rejection method when the shape parameter $\alpha \geq 1$, and the Ahrens and Dieter (1974) transformation otherwise. Reliable implementations are available in mainstream libraries (e.g. `PyTorch`). For the NIG model, the subordinator follows an IG distribution so exact sampling in this case relies on the Michael et al. (1976) algorithm, see also Cont and Tankov (2004, Algorithm 6.9). For completeness, the procedure is recalled in Algorithm 1.



(a) Gamma subordinator: small vs. large κ



(b) Inverse Gaussian subordinator: small vs. large κ

Figure 2: Paths from the Gamma and the Inverse Gaussian subordinators.

Algorithm 1 Exact sampling of $G \sim \mathrm{IG}(\mu, \lambda)$

Require: $\mu > 0$, $\lambda > 0$

Sample $v \sim \mathcal{N}(0, 1)$, set $y \leftarrow v^2$

Compute

$$x \;\leftarrow\; \mu \;+\; \frac{\mu^2 y}{2\lambda} \;-\; \frac{\mu}{2\lambda}\sqrt{4\mu\lambda y + \mu^2 y^2}$$

Sample $u \sim \mathcal{U}[0, 1]$

Set $G \leftarrow x$ if $u \leq \mu/(\mu + x)$; else $G \leftarrow \mu^2/x$

return G

The training datasets for the VG and NIG models are drawn from parameter domains chosen to balance financial realism, numerical stability, and coverage of salient regimes. For the VG model, we sample

$$\theta \in [-0.4,\, 0.2], \qquad \kappa \in [0.05,\, 0.4], \qquad \sigma \in [0.1,\, 0.4].$$

These intervals reflect empirical calibrations (e.g. S&P 500 options in Madan, Carr, et al. (1998)), while extending toward more active jump regimes for robustness; see also Raible (2000). For the NIG model, parameters are drawn as

$$\theta \in [-0.4,\, 0.2], \qquad \kappa \in [0.1,\, 0.6], \qquad \sigma \in [0.1,\, 0.5].$$

These ranges cover typical calibrations (Barndorff-Nielsen 1997) and allow for heavier tails than VG at comparable variance levels, consistent with empirical reviews such as Raible (2000). Maturities are restricted to $T \in [25/365,\, 190/365]$, where non-Gaussian features materially affect prices.

Remark 4.2. The focus on short to medium maturities is motivated by the central limit theorem: subordinated models are infinitely divisible (Sato 1999), and at long horizons their marginals converge towards Gaussian laws, so skewness and kurtosis progressively vanish. For longer maturities, stochastic volatility models such as Heston (1993) provide a more suitable description.

Martingale adjustment. Under the risk–neutral measure, the discounted asset price $S_t = \exp(X_t)$ must satisfy the martingale condition

$$\mathbb{E}[S_t] = \mathbb{E}[e^{X_t}] = 1, \qquad \forall t \geq 0.$$

For a Lévy process X, one has

$$\mathbb{E}[e^{uX_t}] = e^{t\psi(u)}, \qquad \psi(u) = \log \mathbb{E}[e^{uX_1}],$$

where ψ is the cumulant generating function of the unit increment X_1. In particular,

$$\mathbb{E}[e^{X_t}] = e^{t\psi(1)},$$

so the raw process X_t does not in general satisfy the martingale property.

To correct this, we subtract a deterministic drift that depends on the model parameters and the maturity. Given $\Theta = (\theta, \kappa, \sigma, t)$, define

$$\tilde{X}_t = X_t - w(\Theta), \qquad w(\Theta) = t\,\psi(1; \theta, \kappa, \sigma).$$

Then

$$\mathbb{E}[e^{\tilde{X}_t}] = e^{-w(\Theta)}\,\mathbb{E}[e^{X_t}] = e^{-t\psi(1)}\,e^{t\psi(1)} = 1,$$

so that $\tilde{X}_t$ produces a martingale discounted price.

For the VG model (Madan and Seneta 1990), the CGF is

$$\psi(u) = -\tfrac{1}{\kappa}\log\big(1 - \kappa\theta u - \tfrac{1}{2}\kappa\sigma^2 u^2\big),$$

so that

$$w(\Theta) = -\tfrac{t}{\kappa}\log\big(1 - \kappa\theta - \tfrac{1}{2}\kappa\sigma^2\big),$$

which is well-defined whenever $1 - \kappa\theta - \frac{1}{2}\kappa\sigma^2 > 0$.

For the NIG model (Eberlein and Prause 2002), one has

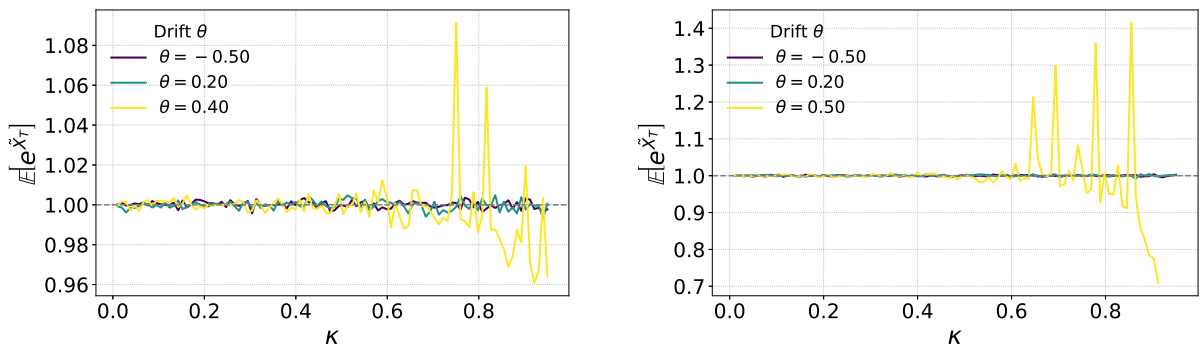
$$\psi(u) = \tfrac{1}{\kappa}\Big(1 - \sqrt{1 - 2\kappa\theta u - \kappa\sigma^2 u^2}\Big),$$

hence

$$w(\Theta) = \tfrac{t}{\kappa}\Big(1 - \sqrt{1 - 2\kappa\theta - \kappa\sigma^2}\Big),$$

with the admissibility condition $1 - 2\kappa\theta - \kappa\sigma^2 > 0$ ensuring finiteness of the exponential moment.

In practice, GANs are trained on the raw processes X_t, and the martingale adjustment is applied afterwards so that all reported quantities refer to the corrected process $\tilde{X}_t = X_t - w(\Theta)$. Within the admissible parameter domains, these corrections are $\mathcal{C}^\infty$ transformations of X_t, and therefore fully compatible with AAD. Figures 3 illustrate that where deviations from 1 reflect numerical instability of the martingale adjustment. Although the correction enforces $\mathbb{E}[e^{\tilde{X}_T}] = 1$ in theory, numerical instabilities appear for large drift values (e.g. $\theta \gtrsim 0.2$), especially in the VG case. This further supports the choice of the compact training domain of θ as specified earlier.



(a) NIG

(b) VG

Figure 3: Estimated martingale expectation $\mathbb{E}\left[e^{\tilde{X}_T}\right]$ from 100,000 Monte Carlo paths across κ for different drifts θ with fixed $\sigma = 0.3$ and $T = 2.0$.

Networks architectures. Although VG and NIG share the same subordination framework, we use different network capacities while keeping the overall design consistent with Section 3.1. For a detailed description of the architecture, the reader is referred to that section; here we only summarize the generator and discriminator by their parameters.

- *NIG.*

$$G : (p{=}4, d{=}5, m_\Theta{=}50, m_z{=}100, m{=}150, L_G{=}4)$$

$$D : (p{=}4, m_\Theta{=}50, m_y{=}100, m{=}100, L_D{=}4)$$

- *VG.*

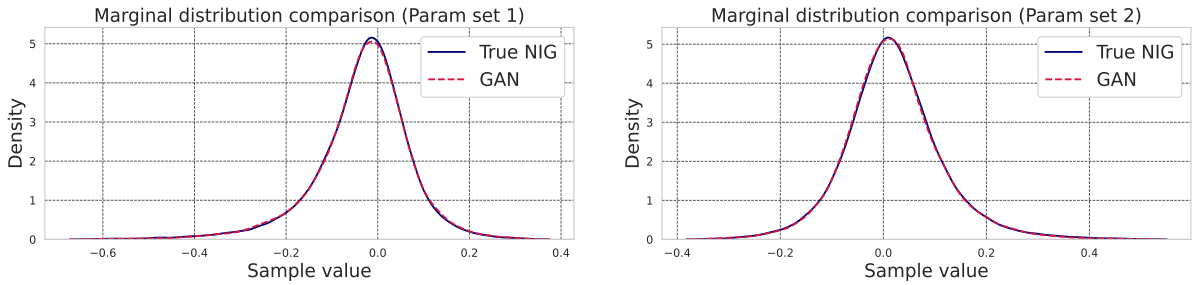
$$G : (p{=}4, d{=}5, m_\Theta{=}150, m_z{=}150, m{=}300, L_G{=}4)$$

$$D : (p{=}4, m_\Theta{=}150, m_y{=}150, m{=}300, L_D{=}4)$$

4.2 Training results

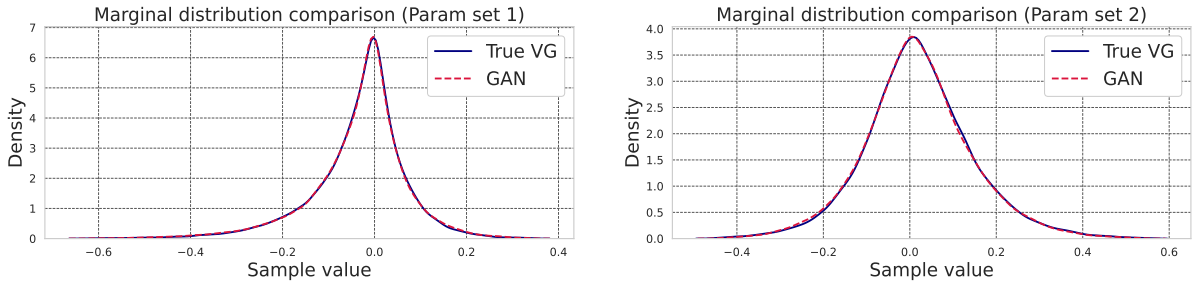
We benchmark the trained generators against exact analytical laws for both NIG and VG models. To build intuition, Figures 4a and 4b display generated densities (red dashed) versus their analytical counterparts (blue) for two representative parameter sets. In both models, the generator succeeds in reproducing not only the central peak but also skewness and heavy tails, with close agreement observed in both symmetric and highly skewed cases.

Beyond the illustrative density plots, we conduct a large-scale evaluation strictly following the protocol defined in Section 3.2. For each of 100,000 parameter configurations, 100,000 samples are generated, martingale-corrected, and compared with exact simulations. The detailed definitions of the reported quantities are provided in Section 3.2, which should be consulted for proper interpretation of the metrics. The results, summarised in Table 2, confirm the high accuracy of the generators: the *mean-preserving property* holds within 10^{-3}, moment deviations remain negligible, and even extreme configurations stay under control. Distributional fit is equally convincing, with KS statistics averaging below 0.01 (maxima 0.04 for NIG and 0.11 for VG) and Wasserstein distances consistently of order 10^{-3}. Taken together, these findings establish conditional GANs as a dependable approach for generating NIG and VG marginals, faithfully capturing both symmetry and heavy-tailed behaviour across admissible domains.



(a) NIG: GAN-generated (dashed red) versus exact density (solid blue).

Param set	θ	κ	σ	T (days/365)
1	-0.20	0.20	0.25	60/365
2	0.05	0.35	0.15	150/365



(b) VG: GAN-generated (dashed red) versus exact density (solid blue).

Param set	θ	κ	σ	T (days/365)
1	-0.20	0.20	0.25	60/365
2	0.05	0.20	0.20	150/365

Figure 4: Comparison between GAN-generated densities (red dashed) and exact densities (solid blue).

Table 2: Metrics over 100,000 parameter sets for NIG and VG models.

NIG model

	$\bar{\mu}_i^{\text{true}}$	$\bar{\mu}_i^{\text{gen}}$	e_i^{μ}	e_i^{σ}	$\tilde{e}_i^{\mu}$	$\tilde{e}_i^{\sigma}$	KS_i	$W_{1,i}$
$\mathbb{E}[\cdot]$	-0.00003	0.00013	0.00098	0.00177	0.09833	1.17662	0.00611	0.00185
$\text{std}(\cdot)$	0.00056	0.00126	0.00097	0.00195	0.09719	1.25140	0.00283	0.00106
$\sup_i \lvert \cdot \rvert$	0.00676	0.00988	0.01179	0.02360	1.18139	18.98946	0.04002	0.01180

VG model

	$\bar{\mu}_i^{\text{true}}$	$\bar{\mu}_i^{\text{gen}}$	e_i^{μ}	e_i^{σ}	$\tilde{e}_i^{\mu}$	$\tilde{e}_i^{\sigma}$	KS_i	$W_{1,i}$
$\mathbb{E}[\cdot]$	0.00000	-0.00014	0.00109	0.00141	0.10870	0.98674	0.00716	0.00178
$\text{std}(\cdot)$	0.00048	0.00135	0.00095	0.00170	0.09468	0.94259	0.00369	0.00095
$\sup_i \lvert \cdot \rvert$	0.00301	0.00751	0.00799	0.02768	0.79832	12.19209	0.11490	0.00962

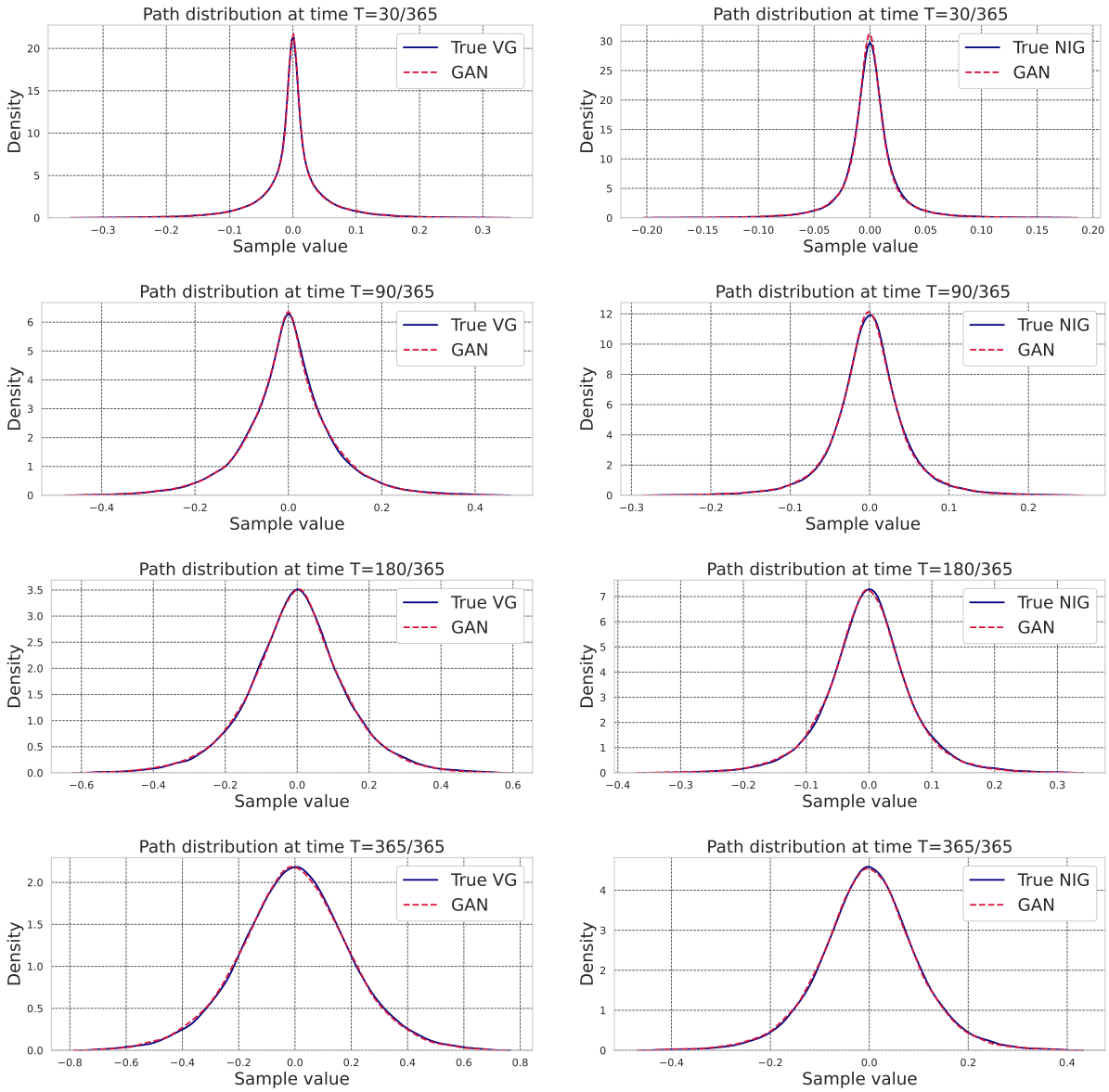


Figure 5: Illustration of GAN-generated marginals X_T (dashed red) versus true distributions (solid blue) for both VG (left, parameters $\theta = 0.00$, $\kappa = 0.25$, $\sigma = 0.20$) and NIG (right, parameters $\theta = 0.00$, $\kappa = 0.50$, $\sigma = 0.10$).

4.3 Pathwise construction

As subordinated Lévy processes have independent and stationary increments, the generator trained on the marginal law $X^{true}(\Theta)$ can be leveraged to synthesize entire trajectories without retraining.

Given a time grid $0 = t_0 < \cdots < t_n$, let $\Delta t_k := t_k - t_{k-1}$ and define $\Theta_k := (\theta, \kappa, \sigma, \Delta t_k)$. At each step k, we simulate an independent noise variable $z_k \sim Z$ and feed it into the VG generator:

$$\Delta X^G_{t_k} = G(\Theta_k, z_k),$$

The path is then recovered by cumulative summation of increments:

$$X^G_{t_k} = \sum_{i=1}^{k} \Delta X^G_{t_i}.$$

If asset prices are required, a martingale drift correction is applied at each step,

$$S_{t_k} = \exp\big(X^G_{t_k} - w(\Theta_k)\big).$$

Because increments are independent, they can be generated fully in parallel, making the method ideally suited to GPU-based Monte Carlo simulation. The resulting simulator is *weak* in the sense that it reproduces the correct marginals at each time t_i. The quality of the reconstructed paths is of course directly tied to that of the generator G: any systematic bias or lack of fidelity in the marginals inevitably accumulates along the trajectory. In practice, however, the generator presented here proves to be highly robust, and all tests conducted show a high level of accuracy across parameter ranges. Figure 5 presents a representative example.

5 Generative networks for Heston and CIR models

5.1 Data generation

The second model on which we train generative networks is the Heston model (Heston 1993), which couples the price of the asset to a stochastic variance driven by a square-root diffusion.

$$\begin{cases} dS_t = \sqrt{v_t}\, S_t\, dW_t^{(1)}, \\ dv_t = \kappa(\bar{v} - v_t)\, dt + \gamma\sqrt{v_t}\, dW_t^{(2)}, \end{cases} \qquad d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\, dt,$$

where v_t follows a CIR process (Cox et al. 1985) with mean-reversion speed $\kappa > 0$, long-run variance $\bar{v} > 0$, volatility of volatility $\gamma > 0$, and correlation $\rho \in [-1, 1]$.

The variance process remains strictly positive if the Feller condition $2\kappa\bar{v} \geq \gamma^2$ holds, under which the origin is inaccessible. When this inequality fails, v_t can hit zero in finite time, leading to stiff dynamics and degenerate stochastic flow regularity as studied by Vostrikova (2009), Abbas-Turki and Lamberton (2014), Mackevičius and Mongirdaitė (2018), and Mishura et al. (2024). Figure 6 illustrates this phenomenon: far from the boundary, CIR trajectories remain smooth and bounded away from zero, while in low-Feller regimes the variance process frequently touches the boundary and exhibits irregular behavior.

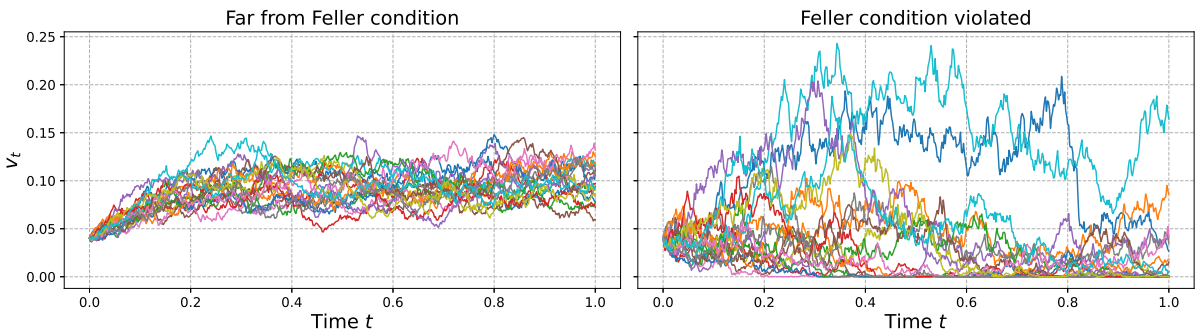


Figure 6: CIR trajectories under different Feller regimes. Left: ratio of Feller comfortably satisfied ($\frac{2\kappa\bar{v}}{\gamma^2} = 30$). Right: ratio of Feller strongly violated ($\frac{2\kappa\bar{v}}{\gamma^2} = 0.08$).

For simulation, exact schemes exist such as the method of Broadie and Kaya (2006), which exploits the noncentral-χ^2 transition of v_T together with Laplace inversion of $\int_0^T v_s\, ds$. While unbiased, repeated Laplace inversions are computationally heavy and ill-suited to GPU pipelines. Instead, we rely on an *almost-exact* scheme: the CIR variance is propagated exactly via its Poisson–χ^2 mixture representation,

$$v_{t+\Delta} \overset{d}{=} c_\Delta\, \chi^2_{d+2N}, \quad N \sim \text{Poisson}\left(\frac{\lambda_\Delta}{2}\right),$$

with

$$c_\Delta = \frac{\gamma^2}{4\kappa}\big(1 - e^{-\kappa\Delta}\big), \qquad d = \frac{4\kappa\bar{v}}{\gamma^2}, \qquad \lambda_\Delta = \frac{4\kappa e^{-\kappa\Delta}}{\gamma^2(1 - e^{-\kappa\Delta})}\, v_t,$$

while the log-price is advanced by a second-order weak scheme of quadratic-exponential type (Andersen 2007):

$$X_{t+\Delta} = X_t + k_0 + k_1 v_t + k_2 v_{t+\Delta} + \sqrt{(1-\rho^2)\, v_t}\, \Delta W_t, \qquad \Delta W_t \sim \mathcal{N}(0, \Delta),$$

with coefficients

$$k_0 = -\frac{\rho\,\kappa\,\bar{v}}{\gamma}\,\Delta, \qquad k_1 = \Big(\frac{\rho\,\kappa}{\gamma} - \tfrac{1}{2}\Big)\Delta - \frac{\rho}{\gamma}, \qquad k_2 = \frac{\rho}{\gamma}.$$

This combination preserves the exact variance marginal while maintaining accurate log-price dynamics at modest computational cost.

The training dataset consists of 5×10^6 i.i.d. samples of X_T, obtained by drawing parameters uniformly from

$$\kappa \in [0.5, 4.0], \qquad \bar{v} \in [0.01, 0.15], \qquad \rho \in [-1.0, -0.3], \qquad v_0 \in [0.01, 0.12],$$

and

$$\gamma \in \Big[\max\big(\sqrt{\kappa\bar{v}/5},\, 0.1\big),\ \min\big(\sqrt{12\kappa\bar{v}},\, 0.8\big)\Big].$$

This parametrization guarantees that the Feller ratio

$$\frac{2\kappa\bar{v}}{\gamma^2} \ \in\ \Big[\tfrac{1}{6},\, 10\Big],$$

and covers both boundary-touching and strictly positive regimes frequently encountered in equity calibration (Cui et al. 2016). Maturities are drawn uniformly from $T \in [50/365,\, 390/365]$ years, with emphasis on short to mid horizons where volatility dynamics are most influential. We accept that training extends to parameter regions where, as shown by Keller-Ressel (2011), the discounted Heston stock ceases to be a true martingale and only remains a supermartingale, as is the case for any positive local martingale. Nevertheless, unlike subordinated jump models such as VG or NIG, no deterministic drift adjustment is required in theory: the generator can be trained directly on log-returns $X_T = \log S_T$, with prices recovered by exponentiation.

Networks architectures. The network follows the general conditional GAN design introduced in Section 3. For architectural details we refer the reader to that section; here we only summarize its structure through the following parameters:

$$G : (p{=}6, d{=}5, m_\Theta{=}150, m_z{=}150, m_G{=}300, L_G{=}4)$$

$$D : (p{=}6, m_\Theta{=}150, m_y{=}150, m_D{=}300, L_D{=}4)$$

5.2 Training results

Figure 7 reports two contrasting situations: one in which the Feller condition is violated, and another where the condition is satisfied, corresponding to a long-maturity setting with high volatility-of-volatility. Throughout these experiments, the generator consistently reproduces the defining features of the Heston model, skewness, fat tails, and steep local gradients, even under challenging parameter conditions.

Beyond the illustrative density plots, we conduct a systematic evaluation following the protocol detailed in Section 3.2. We draw 100,000 Heston parameter configurations uniformly within the training ranges, generate 100,000 samples per configuration, and form prices $S_T = \exp(X_T)$.

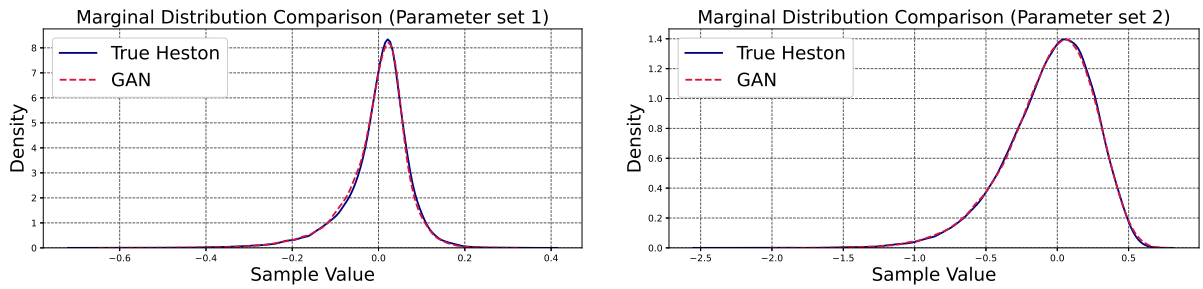


Figure 7: Heston model: comparison of GAN-generated marginals X_T (dashed red) with almost-exact Monte Carlo simulations (solid blue). Left panel: regime violating the Feller condition. Right panel: regime satisfying the Feller condition.

Set	κ	γ	$\bar{v}$	ρ	v_0	T (days/365)	Feller ratio $\frac{2\kappa\bar{v}}{\gamma^2}$
1	1.15	0.59	0.04	-0.49	0.02	90/365	0.26
2	3.86	0.40	0.08	-0.98	0.10	365/365	3.86

For the precise definition of the reported quantities, the reader is referred to Section 3.2. The resulting metrics, summarised in Table 3, show that the generator attains high fidelity to the reference law: the *mean-preserving property* is satisfied up to deviations of order 10^{-3} on average, errors on the first two moments remain limited with even the worst cases under control, and distributional distances are modest (KS statistics average below 0.01, peaking around 0.10, while Wasserstein distances stay of order 10^{-3}). These results confirm that the network accurately reproduces both moment structure and distributional shape of the Heston model across diverse maturities and parameter regimes.

Table 3: Heston model: metrics over 100,000 parameter sets.

	$\bar{\mu}_i^{\text{true}}$	$\bar{\mu}_i^{\text{gen}}$	e_i^{μ}	e_i^{σ}	$\tilde{e}_i^{\mu}$	$\tilde{e}_i^{\sigma}$	KS_i	$W_{1,i}$
$\mathbb{E}[\cdot]$	-0.00143	-0.00131	0.00166	0.00154	0.16639	0.89895	0.00792	0.00261
$\text{std}(\cdot)$	0.00399	0.00411	0.00157	0.00124	0.15744	0.84117	0.00406	0.00146
$\sup_i \lvert\cdot\rvert$	0.02391	0.02384	0.02243	0.01329	2.24295	12.02257	0.07487	0.02243

5.3 Pathwise construction

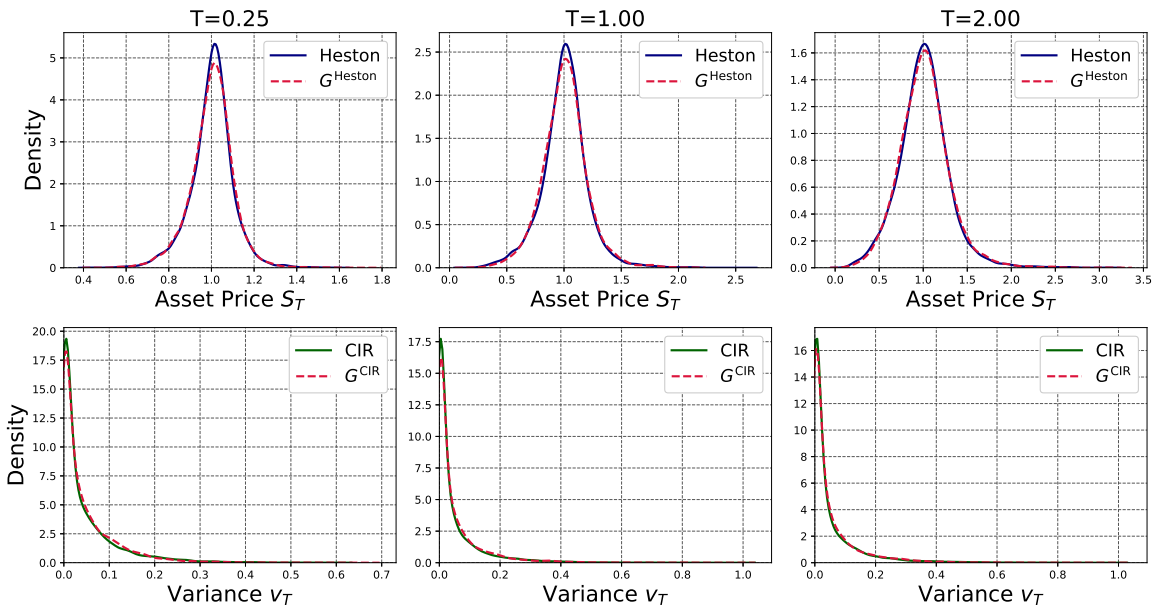
Unlike subordinated Lévy models, the Heston process does not have independent or stationary increments. Nevertheless, full trajectories can be generated step by step by exploiting the Markov property: at each time step, the log–increment is sampled conditionally on the current variance, and the variance itself is advanced through its conditional CIR transition. Given a time grid $0 = t_0 < \cdots < t_n$ and fixed parameters $(\kappa, \gamma, \bar{v}, \rho)$, set $\Delta t_k := t_k - t_{k-1}$ and define

$$\Theta_{v_k,\Delta t_k} := (\kappa, \gamma, \bar{v}, \rho,\ v_k,\ \Delta t_k).$$

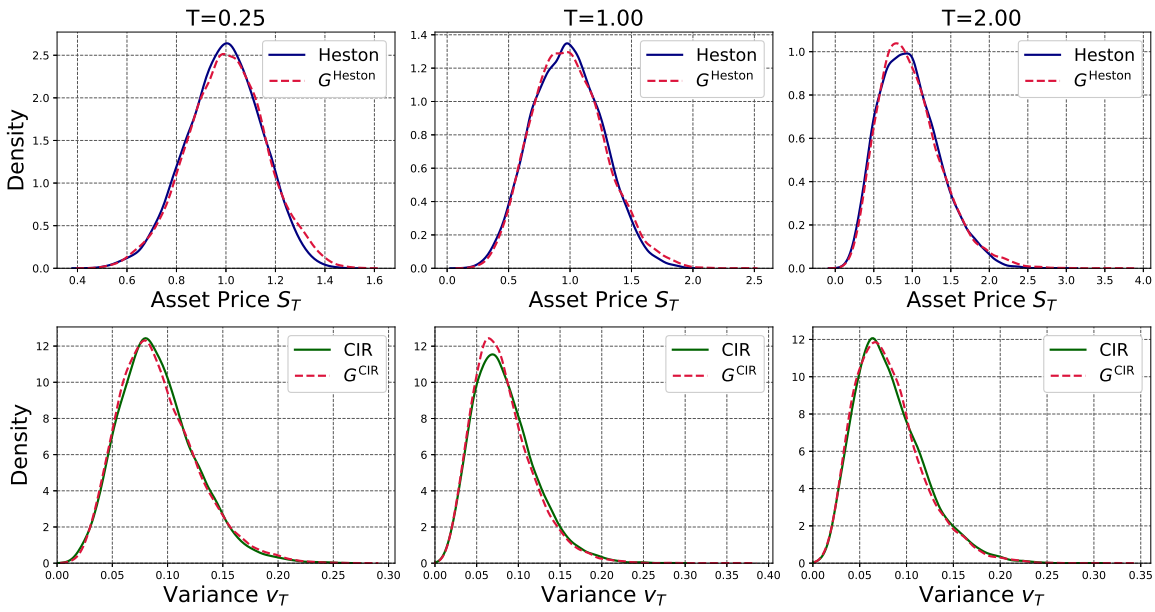
At each step k, we draw independent noises $z_k \sim Z$ and $\tilde{z}_k \sim Z$, and update

$$\Delta X_{t_k}^G = G^{Heston}(\Theta_{v_k,\Delta t_k}, z_k), \qquad v_{k+1} = G^{CIR}(\Theta_{v_k,\Delta t_k}, \tilde{z}_k),$$

where G^{Heston} denotes the generator introduced earlier for Heston log–returns, and G^{CIR} is a dedicated generator trained to reproduce the CIR transition law. Note that for G^{CIR}, the correlation parameter ρ is not required, but we use the same notation $\Theta_{v_k,\Delta t_k}$ for consistency.



Feller-violating regime. $\kappa = 2.15$, $\gamma = 0.80$, $\bar{v} = 0.05$, $\rho = -0.30$, $v_0 = 0.04$. Feller ratio $= 0.34$.



Feller-satisfying regime. $\kappa = 3.00$, $\gamma = 0.31$, $\bar{v} = 0.08$, $\rho = -0.98$, $v_0 = 0.10$. Feller ratio $= 4.99$.

Figure 8: Heston and CIR paths generated by the dual GAN approach. Top panel: regime violating the Feller condition. Bottom panel: regime satisfying the Feller condition.

The log–price path is then recovered by cumulative summation,

$$X^G_{t_k} = \sum_{i=1}^{k} \Delta X^G_{t_i}, \qquad S^G_{t_k} = \exp\big(X^G_{t_k}\big).$$

Because of the variance recursion, the loop is sequential in time but fully parallel across paths on GPU. The resulting simulator is *weak*, matching one–step conditional marginals on the prescribed grid $\{t_k\}$. As in the case of subordinated processes, any sampler of Heston and CIR marginals could replace our GANs, and the trajectory accuracy depends on the quality of these generators. We evaluate the method on the grid $\{0.25, 1.0, 2.0\}$, comparing GAN-generated trajectories with almost-exact Monte Carlo paths. Figure 8 illustrate two regimes with very different Feller ratios (0.34 vs. 4.99), showing that the marginal densities of both the asset and the variance remain well aligned with the benchmark.

6 Generative network for the Rough volatility model

6.1 Data generation

The last model on which we train our generative networks is the rough Bergomi model (Bayer, Friz, et al. 2016). Supported by strong empirical evidence of long memory (Gatheral et al. 2018), the model reproduces the observed roughness of volatility paths, with Hölder exponents well below 0.5. Its dynamics are specified by

$$\begin{cases} dS_t = \sqrt{v_t}\, S_t\, dW^{(1)}_t, \qquad S_0 = 1, \\ v_t = v_0 \, \exp\Big(\eta\, \mathcal{V}_t - \frac{1}{2}\eta^2 t^{2\alpha+1}\Big), \\ \mathcal{V}_t = \sqrt{2\alpha+1} \displaystyle\int_0^t (t-s)^\alpha \, dW^{(2)}_s. \end{cases}$$

Here $W^{(1)}$ and $W^{(2)}$ denote Brownian motions such that $d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\, dt$. The parameter $\eta > 0$ sets the intensity of volatility fluctuations, while roughness is controlled by $H = \alpha + \frac{1}{2} \in (0, 1/2)$. Empirical studies of high-frequency data consistently point to values around $H \approx 0.1$ (Gatheral et al. 2018; El Euch and Rosenbaum 2019), confirming that equity volatility is far rougher than a classical Brownian motion ($H = 1/2$). This rough fractional driver simultaneously generates persistence and violent short-time variations, producing the steep short-maturity implied skews.

For simplicity, the initial forward-variance curve $\xi_0(t)$ is often frozen to a constant level v_0. Although this departs from the most general formulation, it proves robust in practice and preserves the essential features of rough volatility. The price of this realism is numerical complexity: the model is non-Markovian, not a semimartingale, and its Volterra representation requires specialised approximation schemes (Bennedsen et al. 2017). Under the risk-neutral measure, however, the dynamics remain mean-preserving:

$$\mathbb{E}\big[e^{X_t}\big] = S_0, \qquad X_t = \log S_t,$$

so no additional drift correction is needed. A neural generator trained directly on log rough-volatility trajectories therefore inherits martingale consistency by construction.

Simulation is carried out with the hybrid scheme of Bennedsen et al. (2017), which circumvents the singularity of the Volterra kernel by splitting short-range and long-range contributions. The fractional driver

$$\mathcal{V}_t = \sqrt{2\alpha+1} \int_0^t (t-s)^\alpha \, dW^{(2)}_s$$

is approximated on a uniform grid as the sum of two terms: a Gaussian component with exact covariance, and a discrete convolution with a power-law kernel $g(k) \sim k^{\alpha}$. The variance path is then obtained by exponentiation,

$$v_t = v_0 \exp\Big(\eta\, \mathcal{V}_t - \tfrac{1}{2}\eta^2 t^{2\alpha+1}\Big),$$

while the log-price evolves through an Euler discretization,

$$X_{k+1} = X_k - \tfrac{1}{2} v_k\, \Delta t + \sqrt{v_k}\, \Delta W_k^{(1)}, \qquad \Delta W_k^{(1)} \sim \mathcal{N}(0, \Delta t).$$

This convolutional formulation is particularly well-suited to GPUs: it avoids costly Cholesky decompositions, maps naturally to parallel architectures, and yields scalable, high-fidelity samples.

The training dataset consists of five million simulated log-returns, with parameters drawn uniformly, in line with the ranges reported by Bayer, Horvath, et al. (2019), from

$$\alpha \in [-0.49, -0.10], \qquad \eta \in [1.0, 3.0], \qquad \rho \in [-0.95, -0.30], \qquad v_0 \in [0.01, 0.12].$$

Although the roughest features are most pronounced at very short horizons, adversarial training in that regime does not achieve the same level of accuracy across all parameter configurations. We therefore restrict maturities to

$$T \in [25/365,\ 190/365],$$

which still preserve the rough signature while allowing stable learning. The forward-variance curve is fixed as $\xi_0(t) \equiv v_0$, a constant-curve approximation that eases training without sacrificing model consistency.

A crucial distinction with Markovian models is that rBergomi retains memory of the entire past. Knowing (S_t, v_t) at a given time t is not sufficient to characterize the law of future prices. As a consequence, a generator trained on one-step marginals cannot be iterated to produce trajectories. Instead, we train it directly on return distributions across arbitrary intervals,

$$X_T - X_{t_0} = \log\Big(\frac{S_T}{S_{t_0}}\Big),$$

conditioned on $\Theta = (\alpha, \eta, \rho, v_0, t_0, T)$. This interval-based formulation respects the rough structure while transmitting richer temporal information. We take $|T - t_0| \geq 25/365$. Note that v_0 denotes the initial variance at time 0, while t_0 marks the start of the interval; the generator thus conditions jointly on the global initialization and the local horizon.

Networks architectures. The network architecture follows the structured conditioning introduced in Section 3.1. For a full description we refer the reader to that section; here we only summarize its design through the following parameters:

$$G : (p{=}6, d{=}5, m_\Theta{=}150, m_z{=}150, m_G{=}300, L_G{=}4)$$

$$D : (p{=}6, m_\Theta{=}150, m_y{=}150, m_D{=}300, L_D{=}4)$$

6.2 Training results

We benchmark the rough Bergomi generator against high-fidelity Monte Carlo simulations of the hybrid scheme (Bennedsen et al. 2017). Figure 9 displays two representative regimes at $t_0 = 0$. Across the scenarios, the generated distributions (red dashed) remain closely aligned with reference samples (blue), capturing the sharp peaks, skewness, and heavy-tailed behaviour characteristic of rough volatility.

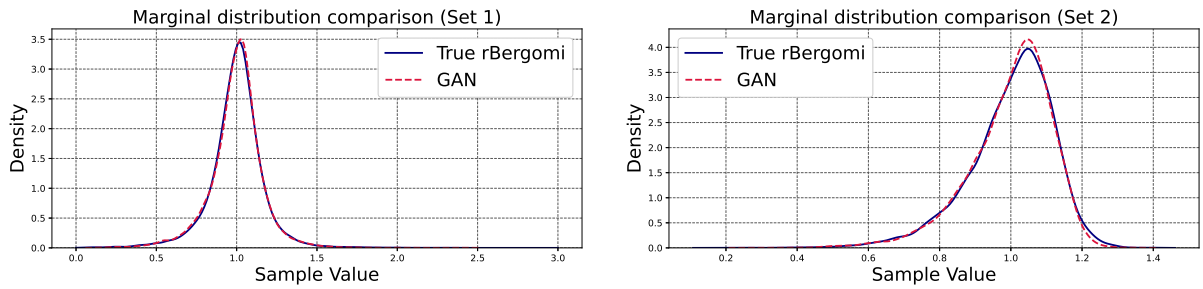


Figure 9: rBergomi model with $t_0 = 0$: comparison of GAN-generated marginals S_T/S_0 (dashed red) with hybrid Monte Carlo simulations (solid blue).

Set	α	η	ρ	v_0	t_0	T (days/365)
1	-0.10	3.00	-0.30	0.07	0.00	180/365
2	-0.45	1.80	-0.90	0.05	0.00	120/365

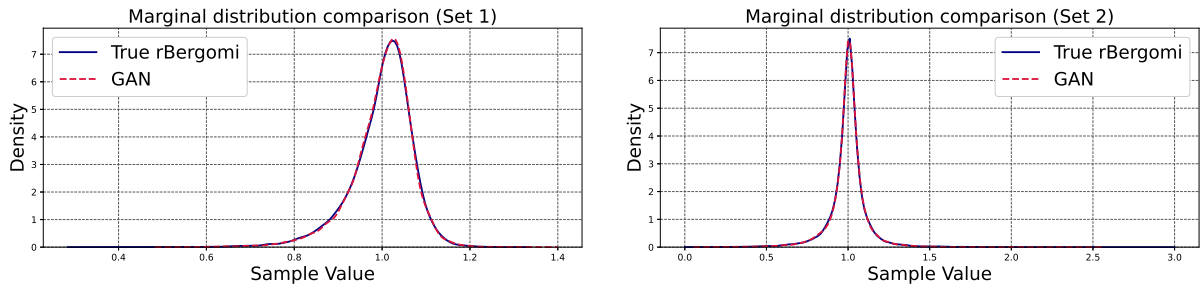


Figure 10: rBergomi model with $t_0 \geq 0$: comparison of GAN-generated marginals S_T/S_{t_0} (dashed red) with hybrid Monte Carlo simulations (solid blue).

Set	α	η	ρ	v_0	$[t_0, T]$
1	-0.40	1.50	-0.80	0.02	[50/365, 150/365]
2	-0.35	2.30	-0.40	0.10	[150/365, 210/365]

We further assess the generator on forward-start intervals $[t_0, T]$ with $t_0 > 0$, figure 10 presents two illustrative regimes. In all such cases, the generated samples (red dashed) remain in close agreement with the Monte Carlo benchmarks (blue), reproducing the skewness and heavy tails typical of rough volatility beyond the initial time.

In order to move beyond the qualitative figures, we conduct a quantitative evaluation over a large test grid of 100,000 parameter configurations, each producing 100,000 samples. The precise definitions of all reported metrics are provided in Section 3.2, and results are distinguished between the two regimes of interest: initial-time marginals ($t_0 = 0$) and shifted intervals ($t_0 > 0$). The numerical outcomes, summarised in Table 4 confirm that the generator delivers a faithful approximation of the rough Bergomi law: the *mean-preserving property* is satisfied within 10^{-3}, moment errors remain minimal, and distributional distances are well controlled (KS averages below 0.01, Wasserstein distances around 10^{-3}). Worst-case deviations are also very reasonable, demonstrating that the generator maintains high accuracy uniformly across parameter regimes—something that typically breaks down if training is pushed to extremely short maturities (1–7 days). Overall, the approach extends robustly to rough volatility, providing accurate marginals across maturities and intervals, while remaining restricted to one-time distributions rather than full path-dependent dynamics.

Table 4: rBergomi: metrics over 100,000 parameter sets.

rBergomi model ($t_0 = 0$)

	$\bar{\mu}_i^{\text{true}}$	$\bar{\mu}_i^{\text{gen}}$	e_i^{μ}	e_i^{σ}	$\tilde{e}_i^{\mu}$	$\tilde{e}_i^{\sigma}$	KS_i	$W_{1,i}$
$\mathbb{E}[\cdot]$	0.00000	0.00010	0.00079	0.00133	0.07870	1.89899	0.01116	0.00148
$\text{std}(\cdot)$	0.00034	0.00096	0.00064	0.00112	0.06442	1.78981	0.00635	0.00068
$\sup_i \lvert\cdot\rvert$	0.00175	0.00696	0.00657	0.00984	0.65706	28.30912	0.10502	0.00666

rBergomi model ($t_0 \geq 0$)

	$\bar{\mu}_i^{\text{true}}$	$\bar{\mu}_i^{\text{gen}}$	e_i^{μ}	e_i^{σ}	$\tilde{e}_i^{\mu}$	$\tilde{e}_i^{\sigma}$	KS_i	$W_{1,i}$
$\mathbb{E}[\cdot]$	0.00000	-0.00012	0.00096	0.00149	0.09553	1.64420	0.00965	0.00174
$\text{std}(\cdot)$	0.00046	0.00119	0.00085	0.00128	0.08457	1.46958	0.00479	0.00091
$\sup_i \lvert\cdot\rvert$	0.00301	0.01051	0.01069	0.01584	1.06884	22.44136	0.07788	0.01089

7 Application on sobolev training

7.1 Gradient supervision

Early neural network approaches to option pricing relied exclusively on L^2 supervision of observed prices (Hutchinson et al. 1994; Garcia and Gençay 1998), a methodology that remains prevalent in more recent contributions (Horvath et al. 2021; Rosenbaum and Zhang 2021). The training signal is confined to the price surface itself, with no supervision on gradients. Formally, the task is to approximate the pricing map

$$\mathbb{R}^p \supset \mathcal{D} \ni \Theta \longmapsto f(\Theta) \in \mathbb{R}.$$

where $\Theta \in \mathcal{D}$ denotes the vector of model and payoff parameters. In this work $\mathcal{D} \subset \mathbb{R}^p$ is a compact domain that includes both model and payoff parameters. We define a random variable Ξ uniformly distributed on $\mathcal{D}$, i.e. $\Xi \sim \mu$ with μ the uniform law on $\mathcal{D}$.

Given a neural approximator $\hat{f}_\phi$ with trainable weights ϕ, the L^2 training objective is

$$\mathcal{L}_{\text{L}^2}(\phi) = \mathbb{E}_{\Xi\sim\mu}\Big[\big(f(\Xi) - \hat{f}_\phi(\Xi)\big)^2\Big].$$

The universal approximation theorem of Cybenko (1989) ensures that sufficiently wide networks can approximate any continuous function in this sense, while Hornik (1991) established that, with smooth activations, the approximation extends to sensitivities.

While these results guarantee the *existence* of accurate approximators for both values and sensitivities, they provide no assurance that standard L^2 training will *recover* faithful gradients in practice. This is explicitly demonstrated by Czarnecki et al. (2017), who show that L^2-only training can yield poor gradient estimates even for simple regression problems. To address this, they propose the *Sobolev training* (ST), in which the loss explicitly enforces agreement between predicted and true gradients alongside value matching:

$$\mathcal{L}_{\text{Sob}}(\phi) = \mathbb{E}_{\Xi\sim\mu}\Big[\big(f(\Xi) - \hat{f}_\phi(\Xi)\big)^2 + \lambda\,\|\nabla_\Theta f(\Xi) - \nabla_\Theta \hat{f}_\phi(\Xi)\|^2_{L^2(\mathbb{R}^n)}\Big],$$

where $\lambda > 0$ balances the relative influence of value and gradient terms.

Empirical studies confirm the benefit of gradient-enhanced supervision. Huge and Savine (2020) shows that incorporating sensitivities into the training objective improves both the accuracy of option prices and the quality of the learned Greeks, while Cocola and Hand (2020) demonstrate that even relatively small neural networks are able to learn values and sensitivities jointly.

The main obstacle is practical: exact pricing gradients $\nabla_\Theta f$ are rarely available in realistic settings (Duffy 2006; Mohamed et al. 2020; Ruf and W. Wang 2020). In our GAN-based

framework, this difficulty is largely bypassed, as the generators are smooth by construction and enable the computation of Monte Carlo estimators of price sensitivities from pathwise derivatives of the payoffs. Since the payoffs are expressed as differentiable functions of the generator output, their derivatives are obtained by AAD with no significant additional computational cost. It nevertheless remains to verify that, in this setting, differentiation with respect to the model parameters can be legitimately interchanged with expectation.

Lemma 7.1. *Let $(\Omega, \mathbb{F}, \mathbb{P})$ be a probability space, $d \in \mathbb{N}^*$, and let $Z : \Omega \to [0,1]^d$ be uniformly distributed on $[0,1]^d$. Let $\mathcal{D} \subset \mathbb{R}^p$ be compact and suppose $G : \mathcal{D} \times [0,1]^d \to \mathbb{R}$ is $\mathcal{C}^1$ in (Θ, z), with*

$$M := \sup_{(\Theta,z)\in\mathcal{D}\times[0,1]^5} \|\nabla_\Theta G(\Theta, z)\| < \infty.$$

Fix $K \in \mathbb{R}$ and define $h(x) = (x-K)_+$. Assume that for each $\Theta \in \mathcal{D}$,

$$\mathbb{P}\big(G(\Theta, Z) = K\big) = 0.$$

Define $f(\Theta) := \mathbb{E}_{\mathbb{P}}\big[h(G(\Theta, Z))\big]$. Then f is differentiable on $\mathcal{D}$ and

$$\nabla_\Theta f(\Theta) = \mathbb{E}_{\mathbb{P}}\left[\mathbf{1}_{\{G(\Theta,Z)>K\}} \nabla_\Theta G(\Theta, Z)\right].$$

Proof. Fix $\Theta \in \mathcal{D}$ and define

$$A_\Theta := \{\omega \in \Omega : \ G(\Theta, Z(\omega)) \neq K\}.$$

By assumption, $\mathbb{P}(A_\Theta) = 1$. Since $h(x) = (x-K)_+$ is differentiable on $\mathbb{R} \setminus \{K\}$ and G is $\mathcal{C}^1$, the map $\Theta' \mapsto h(G(\Theta', Z(\omega)))$ is differentiable at Θ for all $\omega \in A_\Theta$, with

$$\nabla_\Theta\big[h(G(\Theta, Z(\omega)))\big] = \mathbf{1}_{\{G(\Theta,Z(\omega))>K\}} \nabla_\Theta G(\Theta, Z(\omega)).$$

Moreover, the uniform bound

$$\big\|\nabla_\Theta h(G(\Theta, Z(\omega)))\big\| \leq \|\nabla_\Theta G(\Theta, Z(\omega))\| \leq M$$

ensures domination. Differentiation therefore commutes with expectation, yielding the stated result. □

Remark 7.2. In our setting, the latent variable Z is uniformly distributed on $[0,1]^d$. For any fixed $\Theta \in \mathcal{D}$, continuity of $G(\Theta, \cdot)$ implies that $G(\Theta, Z)$ admits a density with respect to Lebesgue measure on $\mathbb{R}$, so that

$$\mathbb{P}(G(\Theta, Z) = K) = 0 \quad \text{for all } K \in \mathbb{R}.$$

The no-atom assumption is therefore automatically satisfied, except in degenerate cases where $G(\Theta, z)$ is independent of z.

As a consequence, for the $\mathcal{C}^\infty$ generators introduced in Section 3, gradients of option prices are obtained directly as expectations:

$$\nabla_\Theta f(\Theta) = \nabla_\Theta \mathbb{E}_z[h(G(\Theta, z))] = \mathbb{E}_z[\nabla_\Theta h(G(\Theta, z))].$$

In practice, both prices and sensitivities are estimated by Monte Carlo averaging:

$$f(\Theta) \approx \frac{1}{N}\sum_{i=1}^N h(G(\Theta, z_i)), \qquad \nabla_\Theta f(\Theta) \approx \frac{1}{N}\sum_{i=1}^N \nabla_\Theta h(G(\Theta, z_i)),$$

where $\{z_i\}_{i=1}^N$ are i.i.d. samples from the latent distribution.

7.2 Numerical results for the Heston model

We now report numerical results in the concrete setting of the Heston model, using the generator trained in Section 5. A broader set of experiments covering all model classes and additional training regimes is presented in Author (2026) Our experimental design relies on three datasets:

- **Training.** We generate 3 million parameter configurations Θ_i, sampled uniformly over the admissible Heston domain. Each configuration is priced using 10^5 Monte Carlo paths. Both option prices $f(\Theta_i)$ and their sensitivities $\nabla_\Theta f(\Theta_i)$ are stored and used for supervision.
- **Validation.** A separate set of 300,000 configurations is computed with 10^6 Monte Carlo paths. These high-precision labels are used to monitor generalisation, and the best checkpoint is selected according to the L^2 validation error.
- **Testing.** Another 300,000 configurations, also priced with 10^6 paths, are reserved exclusively for final evaluation.

Remark 7.3. On a Tesla V100–SXM2–32GB GPU, constructing the training dataset of 3 million price and gradient labels using AAD requires approximately **14 hours**.

The pricing network H is identical across all experiments. It is a fully connected MLP with four hidden layers of width 128, using the smoothed ReLU activation `Softplus`($\beta = 10$, threshold 20). The final affine layer outputs a scalar, followed by a standard `Softplus` to enforce positivity of option prices. Parameter ranges, reported in Table 5, coincide with those used in Section 5, ensuring consistency between the generative prior and the supervised labels.

We consider the following two training regimes:

- $\mathbf{H}_1$ — **Price+gradient supervision**: offline Sobolev training on precomputed price and sensitivity datasets.
- $\mathbf{H}_2$ — **Price-only supervision**: offline training on precomputed price datasets.

Parameter	Range
κ	$[0.5,\ 4.0]$
$\bar{v}$	$[0.01,\ 0.15]$
γ	$\left[\max\left(\sqrt{\kappa\bar{v}/5},\ 0.1\right),\ \min\left(\sqrt{12\kappa\bar{v}},\ 0.8\right)\right]$
ρ	$[-1.0,\ -0.3]$
v_0	$[0.01,\ 0.12]$
T	$[50/365,\ 380/365]$

Table 5: Heston parameter ranges used in training with the adaptive bounds on γ ensure that the Feller ratio $\frac{2\kappa\bar{v}}{\gamma^2}$ remains between 1/6 and 10.

7.2.1 Quantitative comparison: price vs. price+gradient supervision

Figure 11 reports the relative L^1 errors on option price sensitivities for the Heston model, averaged over the 300,000 samples of the test set, and compares the two offline training regimes: *price-only* supervision ($\mathbf{H}_2$) and *price+gradient* Sobolev supervision ($\mathbf{H}_1$).

Price accuracy. Including gradients in the training objective does not degrade price accuracy; on the contrary, it leads to a systematic improvement. On average, price+gradient supervision

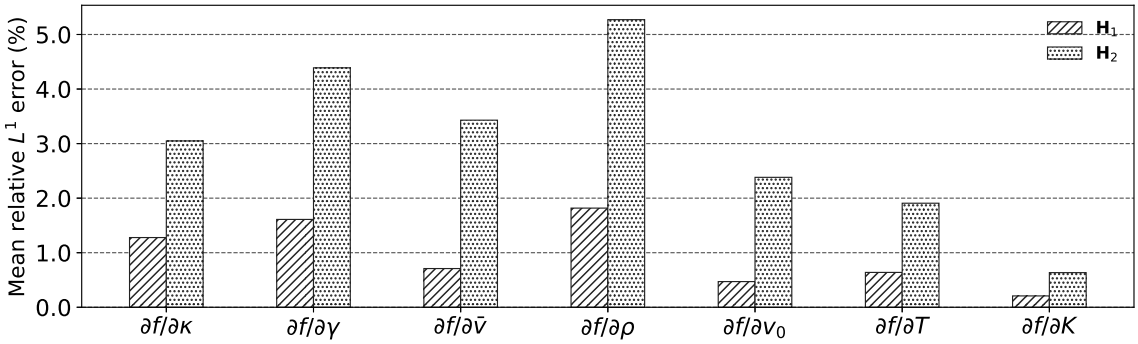


Figure 11: Relative L^1 errors (%) on the price and its sensitivities for the Heston model.

achieves a relative pricing error of 0.1445%, compared with 0.2478% for price-only training. This confirms that Sobolev training preserves, and even slightly enhances, the quality of price approximation.

Gradient accuracy. The advantage of Sobolev supervision becomes markedly more pronounced when examining sensitivities. Across all parameters, learned with price+gradient supervision, gradients are substantially more accurate than those obtained from price-only training. Relative errors remain typically below 2% for κ, γ and ρ, and below 1% for $\bar{v}$, v_0, T and K. By contrast, price-only supervision yields gradient errors in the 3–5% range, with particularly visible gaps on ρ and γ. These results show that while pure price fitting already induces reasonable sensitivities, explicit gradient supervision leads to a clear and systematic reduction of errors.

8 Conclusion

This work has shown that conditional generative adversarial networks provide a reliable and accurate framework for learning the marginal laws of widely used financial models across coherent parameter regimes and realistic maturities. For subordinated Lévy models, Heston, and rough Bergomi, the trained generators consistently reproduce not only first moments but also asymmetry, skewness, and tail behavior, which are essential features for option pricing and risk management. The quality of the fit remains stable across heterogeneous configurations, including regimes that are traditionally challenging for classical simulation methods.

Once trained, the generators define smooth $\mathcal{C}^\infty$ mappings from model parameters and noise to asset returns, entirely free from the structural obstacles that limit the use of automatic adjoint differentiation in historical simulation schemes. In contrast with discretization based approaches, rejection algorithms, or branching constructions, these generators allow exact and immediate access to parameter sensitivities at a fixed computational cost. This represents a substantial methodological shift, as gradients of marginal distributions can be obtained reliably and at scale, without numerical instability or ad hoc approximations.

Coupled with automatic adjoint differentiation, the generators enable the rapid construction of large and faithful databases of prices and sensitivities. The application to Sobolev training of neural pricers illustrates the practical impact of this framework, with a clear improvement in the accuracy of both prices and derivatives compared to classical L^2 training. Altogether, the results establish conditional GANs as a robust and efficient tool for distributional learning in quantitative finance, particularly well suited to gradient based methodologies.

Finally, This dual framework of smooth generator plus Sobolev-trained pricer offers a powerful alternative to classical SDE-based simulation: precise on prices, robust on gradients, scalable on GPUs, and equally well suited to price- and payoff-based Sobolev regimes.

Declarations of Interest

The authors report no conflicts of interest. The authors alone are responsible for the content and writing of the paper.

Acknowledgements

This research has been conducted with the support of the Chair *Capital Markets Tomorrow: Modeling and Computational Issues* under the aegis of the Institut Europlace de Finance, a joint initiative of the Laboratoire de Probabilités, Statistique et Modélisation (LPSM) / Université Paris Cité and Crédit Agricole CIB.

Use of AI Tools

The authors acknowledge that ChatGPT was used as an AI-assisted tool during the preparation of this manuscript to support the drafting of the text and to assist in the search for relevant literature related to the subject. All scientific content, mathematical arguments, numerical experiments, and conclusions have been independently verified by the authors, who remain solely responsible for the accuracy, originality, and integrity of the work.

References

Abbas-Turki, L. and D. Lamberton (2014). "European Options Sensitivity with Respect to the Correlation for Multidimensional Heston Models". In: *International Journal of Theoretical and Applied Finance* 17.03, p. 1450015.

Abi Jaber, E. and O. El Euch (2019). "Multifactor approximation of rough volatility models". In: *SIAM Journal on Financial Mathematics* 10.2, pp. 309–349.

Ahrens, J. H. and U. Dieter (1974). "Computer methods for sampling from gamma, beta, Poisson and binomial distributions". In: *Computing* 12.3, pp. 223–246.

Andersen, L. (2007). *Efficient simulation of the Heston stochastic volatility model.* Working paper, Banc of America Securities. Available at SSRN 946405.

Arjovsky, M., S. Chintala, and L. Bottou (2017). *Wasserstein GAN.* arXiv: 1701.07875.

Author (2026). "Sobolev Training Using GAN-based Models". Manuscript published simultaneously with this paper.

Bachelier, L. (1900). *Théorie de la spéculation.* PhD. Paris: Gauthier-Villars.

Balduzzi, D., M. Frean, L. Leary, J. P. Lewis, K. W.-D. Ma, and B. McWilliams (2017). "The Shattered Gradients Problem: If resnets are the answer, then what is the question?" In: *Proceedings of the 34th International Conference on Machine Learning.* Vol. 70. ICML '17. PMLR, pp. 342–350.

Barndorff-Nielsen, O. E. (1997). "Processes of Normal Inverse Gaussian Type". In: *Finance and Stochastics* 2, pp. 41–68. DOI: 10.1007/s007800050032.

Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018). "Automatic differentiation in machine learning: a survey". In: *Journal of Machine Learning Research* 18.153, pp. 1–43.

Bayer, C., P. K. Friz, and J. Gatheral (2016). "Pricing options under rough volatility". In: *Quantitative Finance* 16.6, pp. 887–904.

Bayer, C., B. Horvath, A. Muguruza, B. Stemper, and M. Tomas (2019). *On Deep Calibration of (Rough) Stochastic Volatility Models.* arXiv: 1908.08806.

Bengio, Y., P. Simard, and P. Frasconi (1994). "Learning long-term dependencies with gradient descent is difficult". In: *IEEE Transactions on Neural Networks* 5.2, pp. 157–166.

Bennedsen, M., A. Lunde, and M. S. Pakkanen (2017). "Hybrid scheme for Brownian semistationary processes". In: *Finance and Stochastics* 21.4, pp. 931–965.

Black, F. and M. Scholes (1973). "The Pricing of Options and Corporate Liabilities". In: *Journal of Political Economy* 81.3, pp. 637–654. DOI: 10.1086/260062.

Bond-Taylor, S., A. Leach, Y. Long, and C. G. Willcocks (2021). "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence.* DOI: 10.1109/TPAMI.2021.3116668.

Broadie, M. and O. Kaya (2006). "Exact simulation of stochastic volatility and other affine jump diffusion processes". In: *Operations Research* 54.2, pp. 217–231.

Castelló, A., S. Barrachina, M. F. Dolz, E. S. Quintana-Ortí, P. San Juan, and A. E. Tomás (2022). "High performance and energy efficient inference for deep learning on multicore ARM processors using general optimization techniques and BLIS". In: *Journal of Systems Architecture* 125, p. 102459. ISSN: 1383-7621.

Chen, Z., Y. Deng, Y. Wu, Q. Gu, and Y. Li (2022). *Towards Understanding Mixture of Experts in Deep Learning.* arXiv: 2208.02813.

Cocola, J. and P. Hand (2020). "Global Convergence of Sobolev Training for Overparameterized Neural Networks". In: International Conference on Machine Learning, Optimization, and Data Science, pp. 574–586.

Cont, R. and P. Tankov (2004). *Financial Modelling with Jump Processes.* 1st ed. London: CRC Press.

Cox, J. C., J. E. Ingersoll, and S. A. Ross (1985). "A Theory of the Term Structure of Interest Rates". In: *Econometrica* 53.2, pp. 385–407.

Cui, Y., S. del Baño Rollin, and G. Germano (2016). "Full and fast calibration of the Heston stochastic volatility model". In: *Quantitative Finance* 17.1, pp. 63–77.

Cybenko, G. (1989). "Approximation by superpositions of a sigmoidal function". In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314.

Czarnecki, W. M., S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu (2017). "Sobolev training for neural networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30.

Duffy, D. J. (2006). *Finite Difference Methods in Financial Engineering: A Partial Differential Equation Approach*. Wiley Finance. John Wiley & Sons.

Eberlein, E. and U. Keller (1995). "Application of generalized hyperbolic Lévy motions to finance". In: *Bernoulli*, pp. 281–299.

Eberlein, E. and K. Prause (2002). "The Generalized Hyperbolic Model: Financial Derivatives and Risk Measures". In: *Mathematical Finance—Bachelier Congress 2000*. Ed. by H. Geman, D. Madan, S. Pliska, and T. Vorst. Springer Finance. Springer, pp. 245–267.

El Euch, O. and M. Rosenbaum (2018). "Perfect hedging in rough Heston models". In: *The Annals of Applied Probability* 28.6, pp. 3813–3856. DOI: 10.1214/18-AAP1408.

— (2019). "Characteristic function of rough Heston models". In: *Mathematical Finance* 29.1, pp. 3–38.

Friz, P. K., W. Salkeld, and T. Wagenhofer (2025). "Weak Error Estimates for Rough Volatility Models". In: *The Annals of Applied Probability* 35.1, pp. 64–98.

Garcia, R. and R. Gençay (Nov. 1998). *Pricing and Hedging Derivative Securities with Neural Networks and a Homogeneity Hint*. Scientific Series 98s-35. Montréal: CIRANO.

Gatheral, J., T. Jaisson, and M. Rosenbaum (2018). "Volatility is rough". In: *Quantitative Finance* 18.6, pp. 933–949.

Glorot, X. and Y. Bengio (2010). "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 9. JMLR Workshop and Conference Proceedings, pp. 249–256.

Goldberg, D. (1991). "What Every Computer Scientist Should Know About Floating-Point Arithmetic". In: *ACM Computing Surveys* 23.1, pp. 5–48.

Goodfellow, I. J., J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio (2014). "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems* 27.

Gremse, F., A. Höfter, L. Razik, F. Kiessling, and U. Naumann (2016). "GPU-accelerated adjoint algorithmic differentiation". In: *Computer Physics Communications* 200, pp. 300–311. DOI: 10.1016/j.cpc.2015.10.027.

Griewank, A. and A. Walther (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Society for Industrial and Applied Mathematics.

Hager, W. W. (2000). "Runge Kutta methods in optimal control and the transformed adjoint system". In: *Numerische Mathematik* 87.2, pp. 247–282.

He, K., X. Zhang, S. Ren, and J. Sun (2016). "Deep residual learning for image recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 770–778.

Heston, S. L. (1993). "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options". In: *The Review of Financial Studies* 6.2, pp. 327–343.

Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*. 2nd ed. SIAM.

Ho, Jonathan, Ajay Jain, and Pieter Abbeel (2020). "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 6840–6851.

Homescu, C. (2011). *Adjoints and Automatic (Algorithmic) Differentiation in Computational Finance*. arXiv: 1107.1831.

Hornik, K. (1991). "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 2.5, pp. 359–366.

Hornik, K., M. Stinchcombe, and H. White (1989). "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5, pp. 359–366.

Horvath, B., A. Muguruza, and M. Tomas (2021). "Deep learning volatility". In: *Quantitative Finance* 21.11, pp. 1861–1879.

Hückelheim, J., H. Menon, W. Moses, B. Christianson, P. Hovland, and L. Hascoët (2024). "A taxonomy of automatic differentiation pitfalls". In: *WIREs Data Mining and Knowledge Discovery* 14.6, e1555.

Huge, B. and A. Savine (2020). *Differential Machine Learning*. arXiv: 2005.02347.

Hutchinson, J. M., A. W. Lo, and T. Poggio (Apr. 1994). *A Nonparametric Approach to Pricing and Hedging Derivative Securities via Learning Networks*. NBER Working Paper 4718. Cambridge, MA: National Bureau of Economic Research.

Ioffe, S. and C. Szegedy (2015). "Batch normalization: Accelerating deep network training by reducing internal covariate shift". In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Vol. 37. PMLR, pp. 448–456.

Jordan, M. I. and R. A. Jacobs (1994). "Hierarchical mixtures of experts and the EM algorithm". In: *Neural Computation* 6.2, pp. 181–214.

Keller-Ressel, M. (2011). "Moment Explosions and Long-Term Behavior of Affine Stochastic Volatility Models". In: *Mathematical Finance* 21.1, pp. 73–98.

Kidger, P., J. Foster, X. Li, H. Oberhauser, and T. Lyons (2021). "Neural SDEs as Infinite-Dimensional GANs". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, pp. 5453–5464.

Kingma, D. P. and J. Ba (2015). "Adam: A method for stochastic optimization". In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

Kingma, D. P. and M. Welling (2013). *Auto-Encoding Variational Bayes*. arXiv: 1312.6114.

Kobyzev, I., S. Prince, and M. Brubaker (2020). "Normalizing Flows: An Introduction and Review of Current Methods". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11, pp. 3964–3979. DOI: 10.1109/TPAMI.2020.2992934.

Li, X., T.-K. L. Wong, R. T. Q. Chen, and D. Duvenaud (2020). "Scalable Gradients for Stochastic Differential Equations". In: *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 108. PMLR, pp. 3870–3882.

Li, Z., M. Usman, R. Tao, P. Xia, C. Wang, H. Chen, and B. Li (2022). "A Systematic Survey of Regularization and Normalization in GANs". In: *ACM Computing Surveys*.

Lozano, P. D., T. L. Bagén, and J. Vives (2023). *Neural SDEs for Conditional Time Series Generation and the Signature-Wasserstein-1 Metric*. arXiv: 2301.01315.

Mackevičius, V. and G. Mongirdaitė (2018). "On backward Kolmogorov equation related to CIR process". In.

Madan, D. B., P. Carr, and E. C. Chang (1998). "The Variance Gamma process and option pricing". In: *European Finance Review* 2.1, pp. 79–105.

Madan, D. B. and E. Seneta (1990). "The Variance Gamma (V.G.) Model for Share Market Returns". In: *The Journal of Business* 63.4, pp. 511–524.

Marsaglia, G. and W. W. Tsang (2000). "A simple method for generating gamma variables". In: *ACM Transactions on Mathematical Software (TOMS)* 26.3, pp. 363–372.

Michael, J. R., W. R. Schucany, and R. W. Haas (1976). "Generating random variates using transformations with multiple roots". In: *The American Statistician* 30.2, pp. 88–90.

Mirza, M. and S. Osindero (2014). *Conditional Generative Adversarial Nets*. arXiv: 1411.1784.

Mishura, Y., A. Pilipenko, and A. Yurchenko-Tytarenko (2024). "Low-dimensional Cox–Ingersoll–Ross process". In: *Stochastics: An International Journal of Probability and Stochastic Processes*. Online ahead of print.

Mohamed, S., M. Rosca, M. Figurnov, and A. Mnih (2020). "Monte Carlo Gradient Estimation in Machine Learning". In: *Journal of Machine Learning Research* 21.132, pp. 1–62.

Rahaman, N., A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, and A. Courville (2019). "On the Spectral Bias of Neural Networks". In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

Raible, S. (2000). "Lévy Processes in Finance: Theory, Numerics, and Empirical Facts". Ph.D. thesis. Freiburg, Germany: University of Freiburg.

Ramdas, A., N. García Trillos, and M. Cuturi (2017). "On Wasserstein Two-Sample Testing and Related Families of Nonparametric Tests". In: *Entropy* 19.2, p. 47.

Rhijn, J. van, C. W. Oosterlee, L. A. Grzelak, and S. Liu (2023). "Monte Carlo Simulation of SDEs using GANs". In: *Japan Journal of Industrial and Applied Mathematics* 40.3, pp. 1359–1390. DOI: 10.1007/s13160-022-00534-x.

Rosenbaum, M. and J. Zhang (2021). *Deep Calibration of the Quadratic Rough Heston Model*. arXiv: 2107.01611.

Ruf, J. and W. Wang (2020). "Neural networks for option pricing and hedging: a literature review". In: *Journal of Computational Finance* 26.4, pp. 1–48.

Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). "Learning representations by back-propagating errors". In: *Nature* 323.6088, pp. 533–536. DOI: 10.1038/323533a0.

Sato, K.-i. (1999). *Lévy Processes and Infinitely Divisible Distributions*. Vol. 68. Cambridge Studies in Advanced Mathematics. Cambridge University Press.

Shi, S., Q. Wang, P. Xu, and X. Chu (2016). "Benchmarking State-of-the-Art Deep Learning Software Tools". In: *Proceedings of the 7th International Conference on Cloud Computing and Big Data*. IEEE.

Vostrikova, L. (2009). "On regularity properties of Bessel flow". In: *Journal of Applied Probability* 46.1, pp. 244–256.

Wang, Y. E., G. Y. Wei, and D. Brooks (2019). *Benchmarking TPU, GPU, and CPU Platforms for Deep Learning*. arXiv: 1907.10701.

Wiese, M., R. Knobloch, R. Korn, and P. Kretschmer (2020). "Quant GANs: Deep generation of financial time series". In: *Quantitative Finance* 20.9, pp. 1419–1440.

Wilkinson, J. H. (1963). *Rounding Errors in Algebraic Processes*. Reprinted by Dover Publications. Prentice Hall.